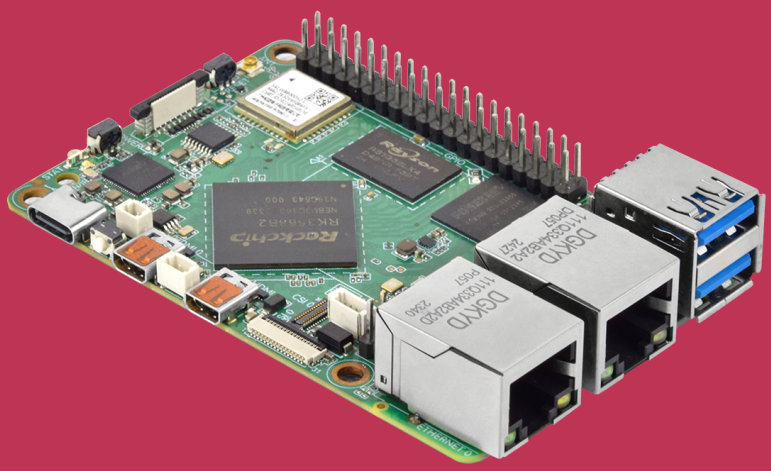
CokePi CPG-3568 用户手册

Published March 2025



免责声明

本文档按“现状”提供，不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

Contents

1 CPG-3568 的基本特性 5
1.1 什么是可乐派 CPG-3568 . . . 5
1.2 可乐派 CPG-3568 的用途 . . . 5
1.3 CPG-3568 的硬件特性 . . . 6
1.4 CPG-3568 的顶层视图和底层视图 . . . 7
1.5 CPG-3568 的主板尺寸图 . . . 8

2 主板使用介绍 9
2.1 准备需要的配件 . . . 9
2.2 下载镜像和相关资料 . . . 14
2.3 烧写系统镜像的方法 . . . 16
2.3.1 镜像获取 . . . 16
2.3.2 镜像命名规则 . . . 16
2.3.3 Windows 平台烧录镜像 . . . 17
2.4 启动可乐派主板 . . . 24
2.5 调试串口的使用方法 . . . 25
2.5.1 调试串口的连接说明 . . . 25
2.5.2 Ubuntu 平台调试串口的使用方法 . . . 27
2.5.3 Windows 平台调试串口的使用方法 . . . 31
2.6 使用可乐派主板 40-Pin 接口中的 5v 引脚供电说明 . . . 35

3 Ubuntu LXQt 桌面系统使用说明 37
3.1 本手册 Linux 命令格式说明 . . . 37
3.2 Linux 系统登录说明 . . . 37
3.2.1 系统默认登录账号和密码 . . . 37
3.2.2 Linux 系统终端登录的方法 . . . 37
3.2.3 Linux 桌面版系统自动登录说明 . . . 38
3.3 板载 LED 灯测试说明 . . . 39
3.4 网络连接测试 . . . 41
3.4.1 以太网口测试 . . . 41
3.4.2 WiFi 连接测试 . . . 42
3.5 蓝牙连接测试 . . . 44

4 Linux SDK 使用说明 48
4.1 SDK 简介 . . . 48
4.2 SDK 开发环境搭建 . . . 48
4.2.1 安装 SDK 依赖的软件包 . . . 48
4.2.2 安装 repo . . . 49
4.2.3 安装 python2.7 . . . 49
4.3 SDK 源码获取 . . . 50

4.3.1 下载地址 . . . 50
4.3.2 校验源码完整性 . . . 50
4.3.3 解压源码 . . . 50
4.4 SDK 更新 . . . 51
4.5 Linux 编译指南 . . . 52
4.5.1 SDK 板级配置准备->RK3568 . . . 52
4.5.2 大版本编译步骤 . . . 52
4.5.3 CPG-3568 系统编译指南 . . . 52
4.6 内核更新 . . . 53
4.6.1 ADB 推送 . . . 54
4.6.2 烧录工具烧写内核 . . . 55

5 Android 11 系统的使用说明 59
5.1 WiFi 的连接测试方法 . . . 59
5.2 蓝牙的测试方法 . . . 61
5.3 8 寸 MIPI 屏幕的使用方法 . . . 66
5.4 OV13850 和 OV13855 MIPI 摄像头的测试方法 . . . 67
5.5 40-Pin 接口测试 . . . 69
5.5.1 GPIO 测试 . . . 70
5.5.2 UART 测试 . . . 71

6 Android SDK 使用说明 75
6.1 SDK 开发环境搭建 . . . 75
6.1.1 安装 SDK 依赖的软件包 . . . 75
6.1.2 安装 repo . . . 75
6.1.3 安装 python2.7 . . . 76
6.2 SDK 源码获取 . . . 76
6.2.1 下载地址 . . . 76
6.2.2 校验源码完整性 . . . 77
6.2.3 解压源码 . . . 77
6.3 SDK 更新补丁 . . . 77
6.4 CPG-3568 系统编译指南 . . . 78
6.4.1 RK3568 编译配置 . . . 78
6.4.2 编译报错及解决办法 . . . 79
6.4.3 编译配置参数文件设定 . . . 79
6.4.4 U-Boot 编译指南 . . . 79
6.4.5 内核编译指南 . . . 79
6.4.6 内核选项配置修改命令 . . . 79
6.4.7 MIPI 输出版本内核编译 . . . 79
6.4.8 HDMI0 输出版本内核编译 . . . 80
6.4.9 HDMI1 输出版本内核编译 . . . 80

6.5 内核更新 80
6.5.1 ADB 推送 80

7 附录 **81**
7.1 用户手册更新历史 81
7.2 镜像更新历史 81

1 CPG-3568 的基本特性

1.1 什么是可乐派 CPG-3568

可乐派 CPG-3568 采用了瑞芯微 RK3568 四核 64 位 Cortex-A55 ARM 处理器，基于高端 22nm 工艺打造，主频最高可达 2.0GHz，集成 ARM Mali-G52 2EE GPU，内嵌高性能 3D 和 2D 图像加速模块，内置 1Tops 算力的 AI 加速器 NPU，可支持 2GB/4GB/8GB 内存，具有 4K 显示处理能力。

可乐派 CPG-3568 引出了相当丰富的接口，包括 HDMI 输出、MIPI 摄像头、FPC 接口的 PCIe2.0、千兆网口、USB2.0、USB3.0 接口和 40-Pin 扩展排针 (I2S、CAN、I2Cx2、UARTx5、PWMx2、SPI、GPIO 接口)。可广泛适用于高端平板、边缘计算、人工智能、云计算、AR/VR、智能安防、智能家居等领域，覆盖 AIoT 各个行业。CPG-3568 支持 Android11、Ubuntu20.04(5.10 内核)、Debian11、LinuxQT 等操作系统。

1.2 可乐派 CPG-3568 的用途

我们可以用它实现：

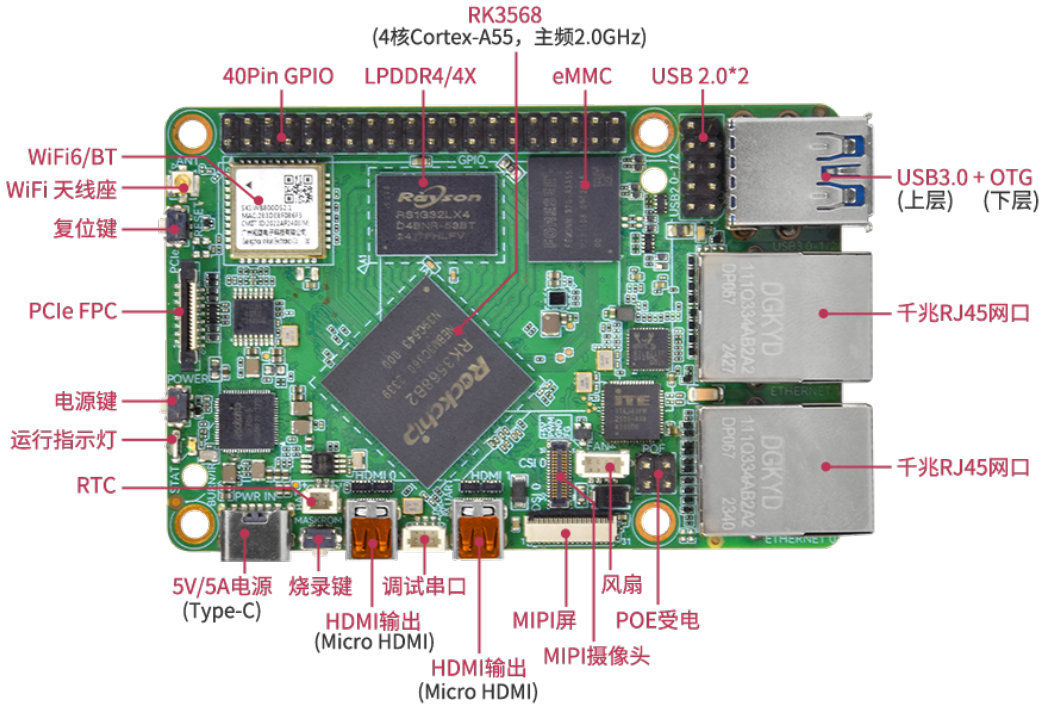
- 一台 Linux 个人桌面计算机
- 一台 Linux 网络服务器
- Android 智能平板
- Android 炫酷游戏机

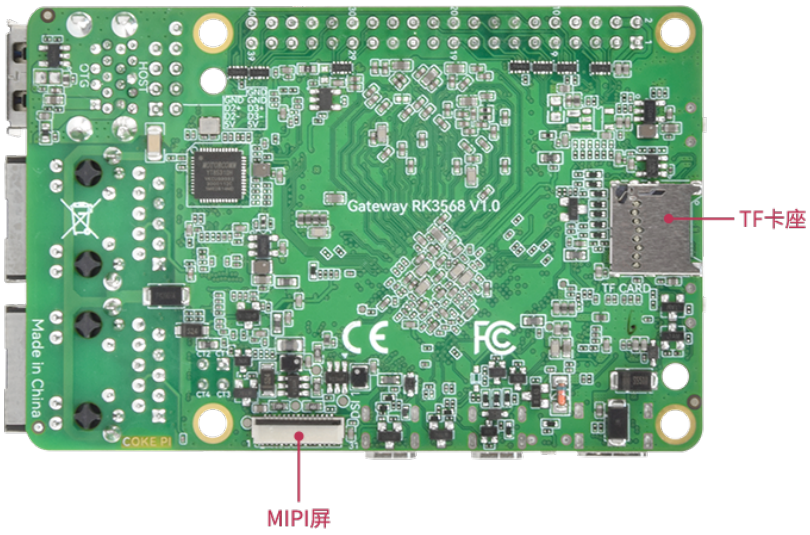
当然，可乐派 CPG-3568 还有其他众多功能。它依托强大的生态系统以及各式各样的扩展配件，能够助力用户轻松达成从创意构思到原型制作，再到批量生产交付的全过程。无论是充满奇思妙想的创客、怀揣梦想的梦想家，还是热衷于自己动手的 DIY 爱好者，可乐派 CPG-3568 都是开展项目的理想平台。

1.3 CPG-3568 的硬件特性

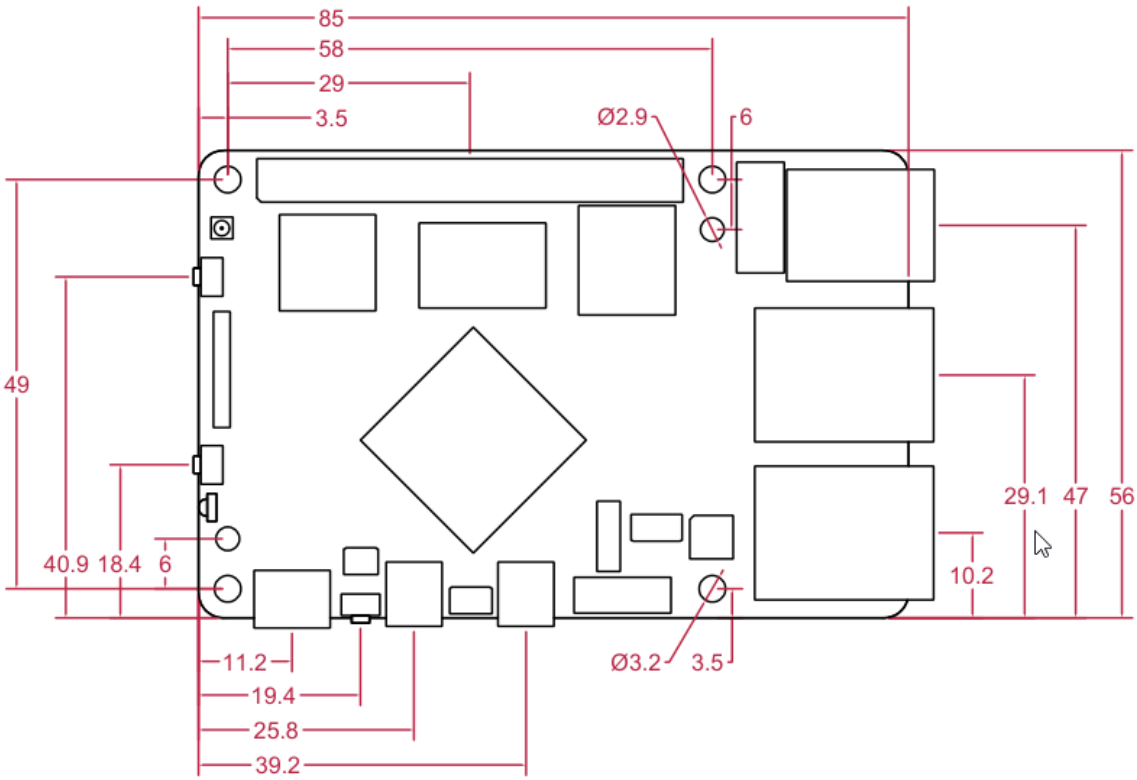
接口/功能	说明
CPU	- Rockchip RK3568 (22nm 工艺打造) - 四核 64 位处理器 - 4 核 Cortex-A55 大小核架构 - 主频最高 2.0GHz
GPU	- 集成 ARM Mali-G52 2EE - 支持 OpenGL ES1.1/2.0/3.2、OpenCL 2.0 和 Vulkan 1.1 - 内嵌高性能 2D 图像加速模块
NPU	- 内置 1Tops 算力的 RKNN NPU AI 加速器 - 支持 Caffe/TensorFlow/TFLite/ONNX/PyTorch/Keras/Darknet 架构模型一键转换
视频输出	- 1 个 Micro HDMI 2.0(HDMI 2.0 TX) 显示接口，最高支持 4K@60FPS 输出 - 1 个 Micro HDMI 2.0(HDMI 1.4 TX) 显示接口，最高支持 4K@30FPS 输出 - 2 个 31-Pin FPC MIPI-DSI 显示接口，最高支持 1920x1200 输出
内存	- 2GB/4GB/8GB（LPDDR4/4x）
摄像头	- 1 路 MIPI CSI 接口，最高支持 1600 万像素的 MIPI 摄像头
板载存储	- 默认标配 16GB EMMC NAND 芯片 (32G|64G|128G 可选)，可扩展至最大 256GB - 16MB SPI 串行 Flash（可选） - TF 卡插座，最高支持 256GB TF 卡
以太网	- 2 路 10/100/1000M 自适应以太网 RJ45 网口 + 4 芯 POE 排针
音频	- HDMI 输出
PCIe 3.0	- 16-Pin PCIe 3.0 FPC 接口可扩展 PCIe 外设
USB 接口	- 2 个 USB 外置横插接口（1 个 USB3.0 和 1 个 USB3.0-OTG）和 2 个内置 USB2.0 插针
40-Pin 扩展排针	- 2.54mm 排针：支持 I2S、CAN、I2Cx2、UARTx5、PWMx2、SPI、GPIO 接口
调试串口	- 1 个 1mm 3-Pin TTL 调试串口
LED 灯	- 绿色工作指示灯和红色指示灯
按键	- 1 个电源键 - 1 个 RECOVERY 烧录键 - 1 个复位键
供电	- 支持 5V-5A 电压直流电源输入（通过 Type-C 座）
支持的操作系统	- Android 11、LinuxQT、Ubuntu 20.04、Debian11

1.4 CPG-3568 的顶层视图和底层视图





1.5 CPG-3568 的主板尺寸图

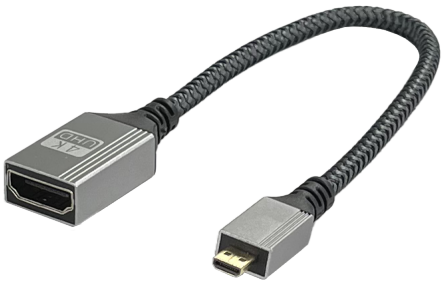


2 主板使用介绍

2.1 准备需要的配件

- 双公头 USB 数据线，用于烧录镜像到 eMMC、使用 ADB 调试

- Micro HDMI 转 HDMI 线，将主板连接到 HDMI 显示器进行显示



- HDMI 连接线，将主板连接到 HDMI 显示器进行显示

- HDMI 接口的显示器

- 8 寸 MIPI 屏幕，用于显示主板的系统界面 (可选)

- 电源适配器和供电线，CPG-3568 建议使用 5V-5A 的 Type-C 电源供电

注意

主板的 Type-C 电源接口不支持 PD 协商功能，只支持固定的 5V 电压输入

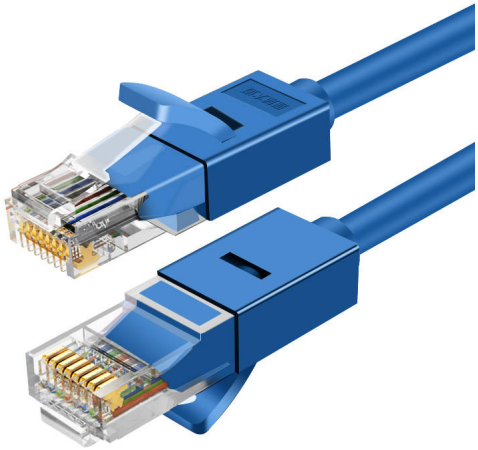
- USB 接口的鼠标和键盘，只要是标准 USB 接口的鼠标和键盘都可以，鼠标和键盘可以用来控制 CokePi 主板

- USB 摄像头

提示

主板插上 Type-C 接口的电源后，40-Pin 排针上的 5V 和 3.3V 引脚就可以直接使用，无需其他设置

- 百兆或者千兆网线，用于将主板连接到因特网

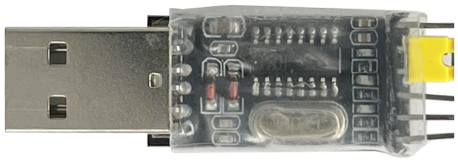


- 标准 IPEX 3dBi 天线

- 1300 万 MIPI 接口的 OV13850 摄像头

- 1300 万 MIPI 接口的 OV13855 摄像头

- 3.3V 的 USB 转 TTL 模块，用于串口调试功能



- SH1.0 转杜邦线 2.54 3P，用于串口调试功能

- 安装有 Windows 或 Ubuntu 操作系统的个人电脑

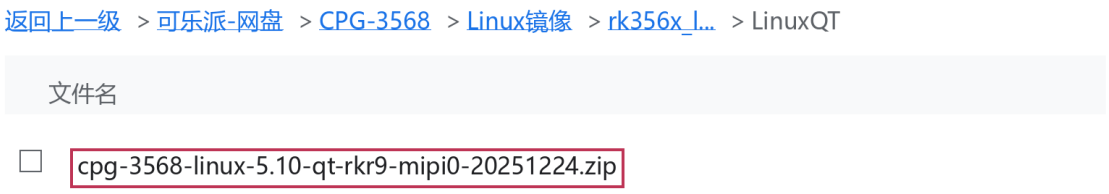
2.2 下载镜像和相关资料

访问百度网盘资源获取 SDK 源码压缩包:
点击进入链接: https://pan.baidu.com/s/1ZCW9zMjR5cWNObJCrRaNlg?pwd=1234
提取码: 1234

- 资料主要包含:

  Android 镜像：在主板上运行安卓 11 操作系统

LinuxQT 镜像：在主板上运行 Buildroot 操作系统



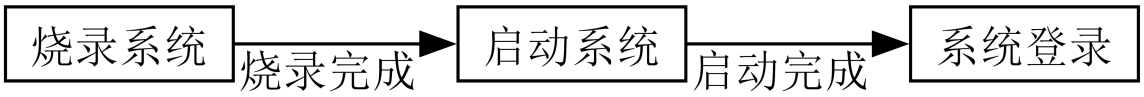
Ubuntu 镜像：在主板上运行 Ubuntu 20.04 操作系统
Debian 镜像：在主板上运行 Debian11 操作系统
Android 源码：用于编译 Android 11 系统内核及根系统
Linux 源码：用于编译 LinuxQT、Debian 系统内核及根系统
用户手册：帮助用户正确使用产品
硬件资料：

- 包含规格书：提供准确的主板参数，以便设计和使用
- 包含原理图：支持调试以及故障排查和验证设计
- 包含机械图：提供准确的尺寸、形状、材质和装配方式，以便制造和组装

工具软件：主要包括主板使用过程中需要用到的软件

2.3 烧写系统镜像的方法

运行板卡时需要三个步骤：

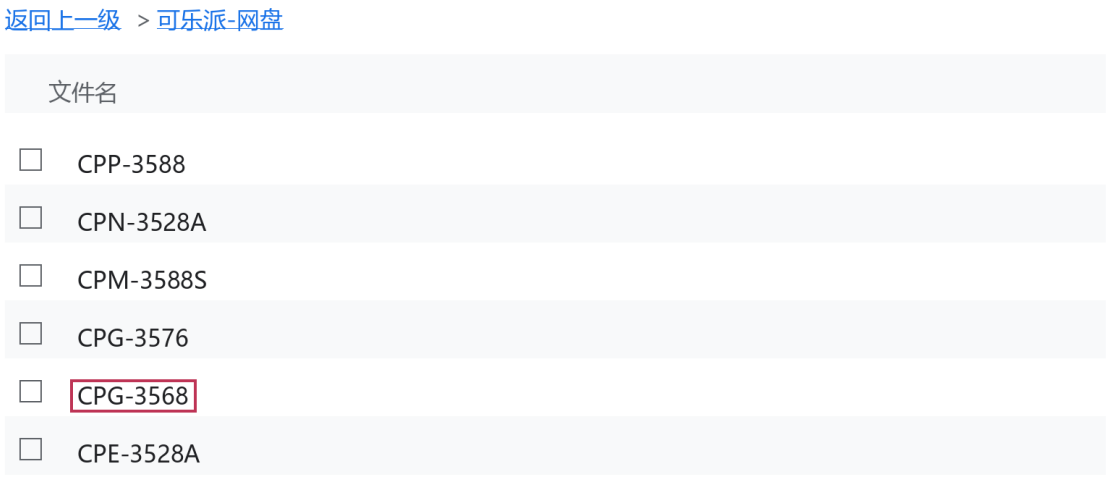


提示

CokePi 系列板卡自带 eMMC 存储，在出厂时都默认烧录了系统镜像，如果不需要烧录新镜像的话可以跳过本章节

2.3.1 镜像获取

请访问百度网盘的资源页面获取系统镜像
点击进入链接: https://pan.baidu.com/s/1ZCW9zMjR5cWNObJCrRaNlg?pwd=1234
提取码: 1234



2.3.2 镜像命名规则

安卓镜像规则：主板名称 + 安卓关键字 + 安卓版本 + SDK 版本 + 显示类型 + 更新时间

Linux 镜像规则：主板名称 + Linux 关键字 + 内核版本 + SDK 版本 + 图形桌面类型 + 显示类型 + 更新时间

以安卓镜像为例：cpg-3568-android-11.0-rkr12-hdmi-20251223.zip

- 主板名称
  cpg-3568：使用 RK3568 处理器的 CokePi 板卡
- 安卓关键字
  android：代表系统使用的安卓系统

- 安卓版本
  11.0：代表安卓版本为 11.0
- SDK 版本
  rkr12：代表 SDK 源码的版本是 r12。注意: 不同的 SDK 版本可能无法兼容
- 显示类型
  mipi：该系统默认支持 MIPI 屏显示 (mipi0: 默认 DSI0)
- 更新时间
  更新时间的格式为 yyyymmdd 即年月日

以 Linux 镜像为例：cpg-3568-linux-5.10-qt-rkr9-mipi0-20251224.zip

- 主板名称
  cpg-3568：使用 RK3568 处理器的 CokePi 板卡
- linux 关键字
  linux：代表系统使用的 Linux 系统
- 内核版本
  5.10：代表内核版本为 5.10
- SDK 版本
  rkr9：代表 SDK 源码的版本是 rkr9。注意: 不同的 SDK 版本可能无法兼容
- 图形桌面类型
  qt：该系统默认支持 linuxqt 系统，使用 qt 桌面
- 显示类型
  mipi：该系统默认支持 MIPI 屏显示
- 更新时间
  更新时间的格式为 yyyymmdd 即年月日

2.3.3 Windows 平台烧录镜像

1. 烧录准备

   硬件：电脑（笔记本）、双公头 USB 数据线、主机/主板、高品质 5V-5A 电源适配器、Type-C 供电线

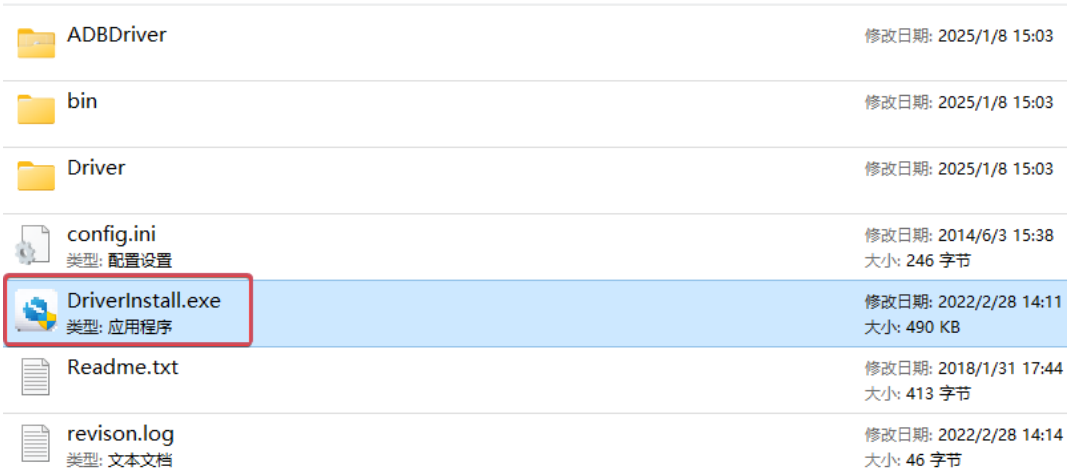
软件：驱动程序 DriverAssitant、系统镜像包 (固件烧录包)RKDevTool

Driverassitant 路径：**可乐派网盘->CPG-3568-> 工具软件**

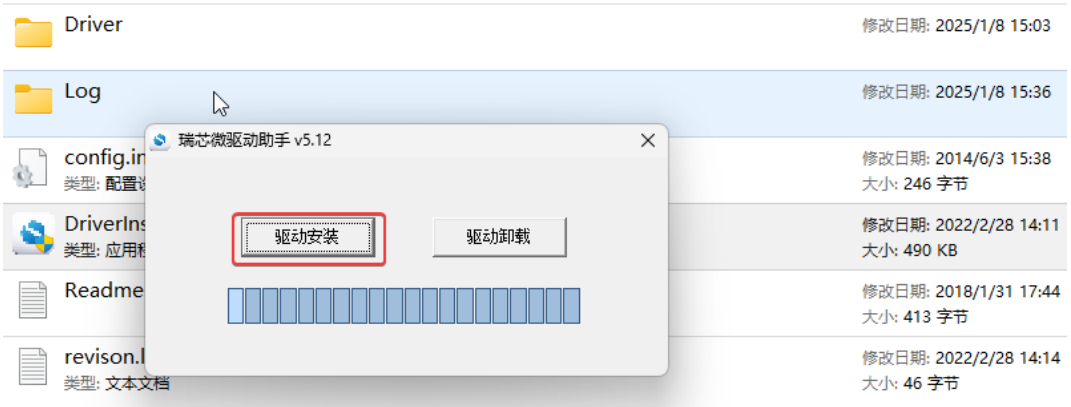
系统镜像包路径 (以安卓系统为例): **可乐派网盘->CPG-3568-> 安卓镜像**

2. 安装 DriverAssitant

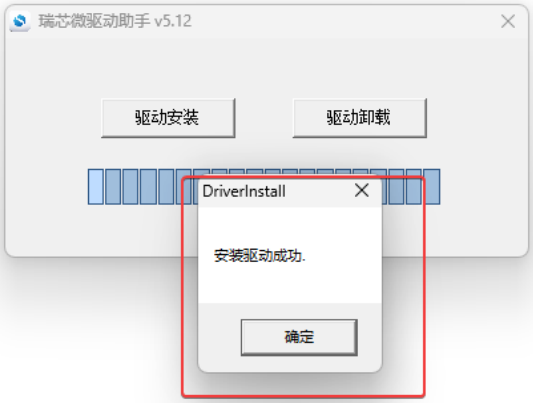
- 解压 DriverAssitant 软件压缩包，双击DriverInstall.exe 进入驱动安装界面



- 点击驱动安装即可开始安装驱动，大约 30s, 驱动安装成功。如果不确定以前是否安装过旧版驱动，先点击驱动卸载移除旧版本驱动程序，再点击驱动安装



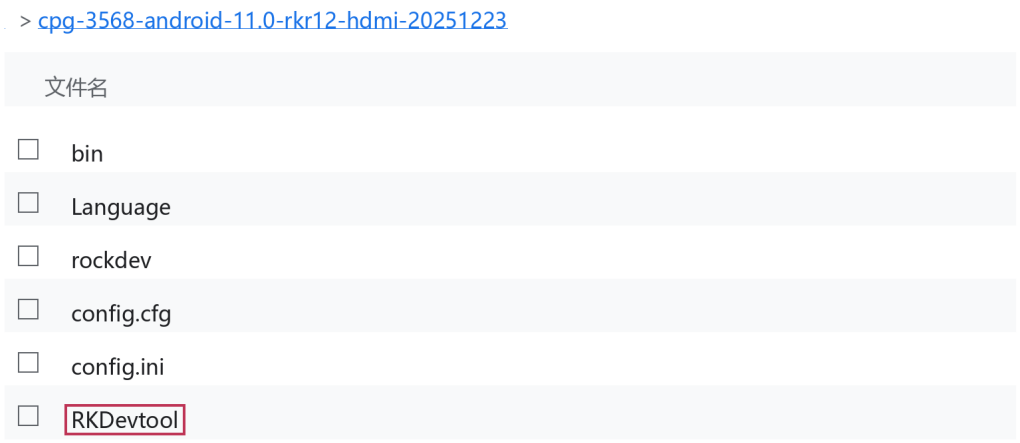
- 等待一段时间后，会弹出窗口提示“安装驱动成功”，然后点击“确定”按钮即可



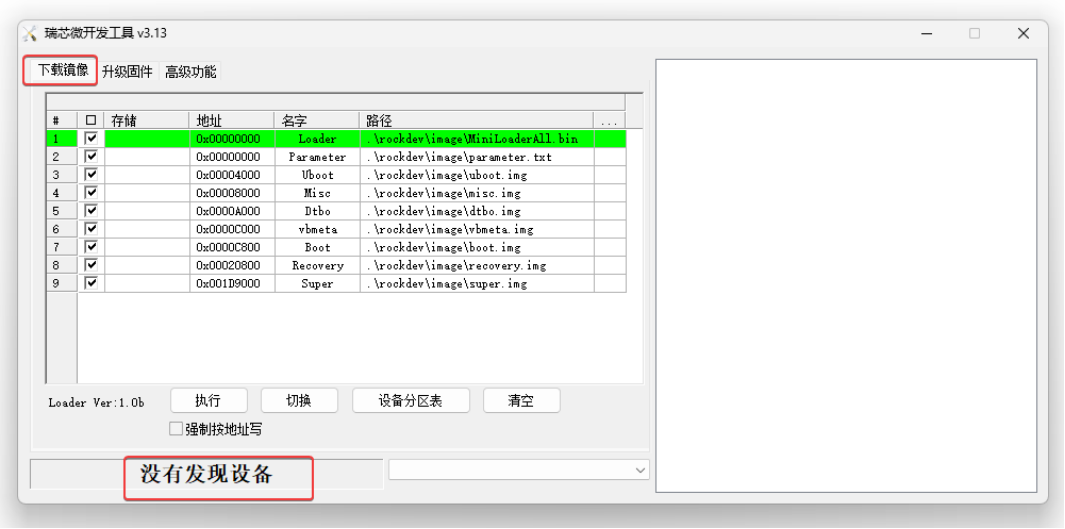
3. 烧录镜像

- 固件烧录包路径 (以安卓 11 系统为例):
  可乐派网盘->CPG-3568-> 安卓镜像->rk356x_android-11.0-mid-rkr12

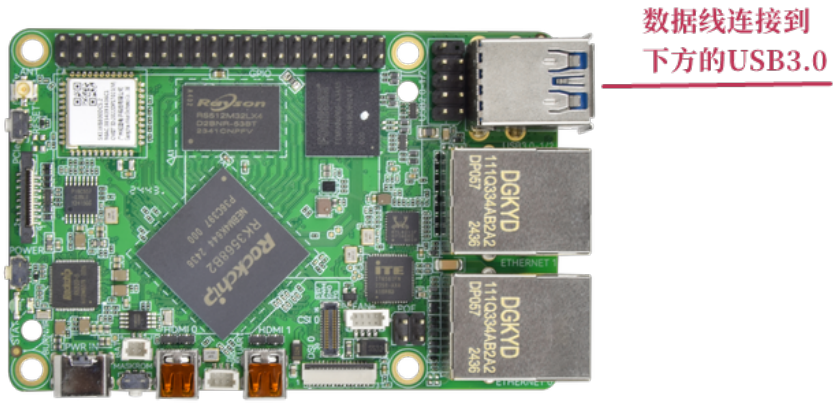
- 解压系统镜像包 (固件烧录包)后无需安装即可使用，双击 RKDevTool.exe 进入软件界面



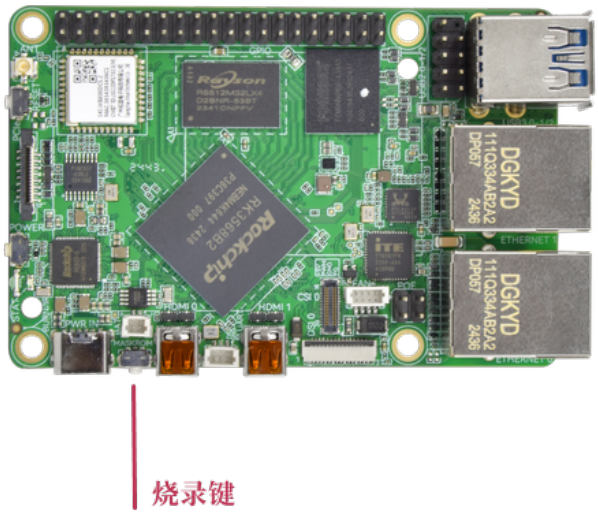
- 软件主要有三大部分，分别是下载镜像、升级固件和高级功能
- 双击 RKDevTool.exe打开烧录工具后，因为电脑此时还没有通过 Type-C 线连接上主板，所以左下角会提示“没有发现设备”



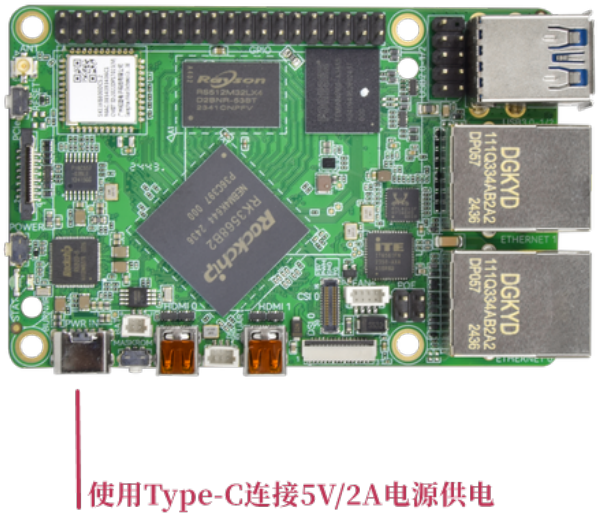
- 首先将所有给主板供电的接线都断开，如电源线，USB 线，烧录线等，确保主板没有连接电源
- 然后通过双公头 USB 数据线连接好 CokePi 主板与 Windows 电脑，主板 OTG 接口的位置如下图所示



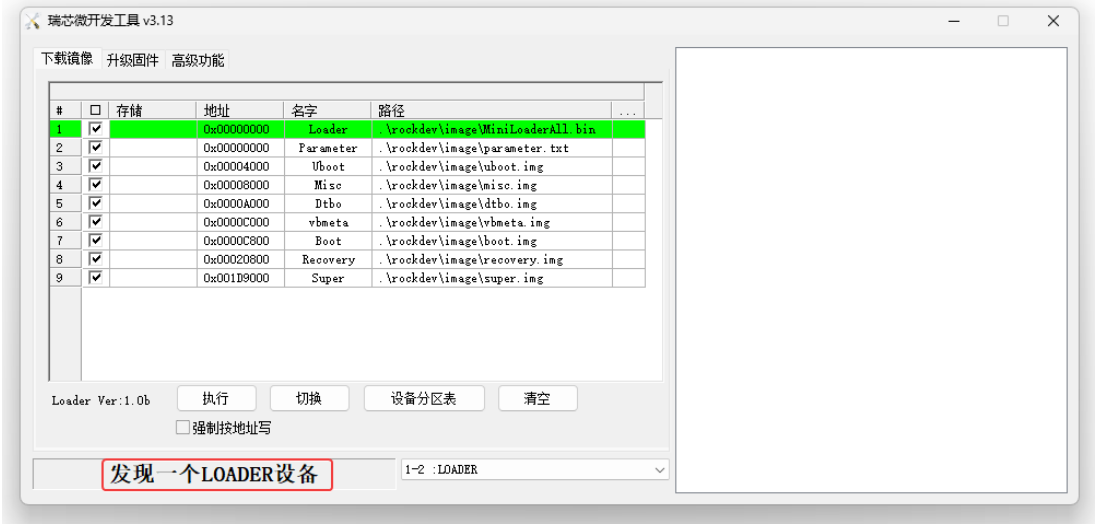
- 然后按住主板的烧录键不放，烧录键在主板的位置如下图所示：



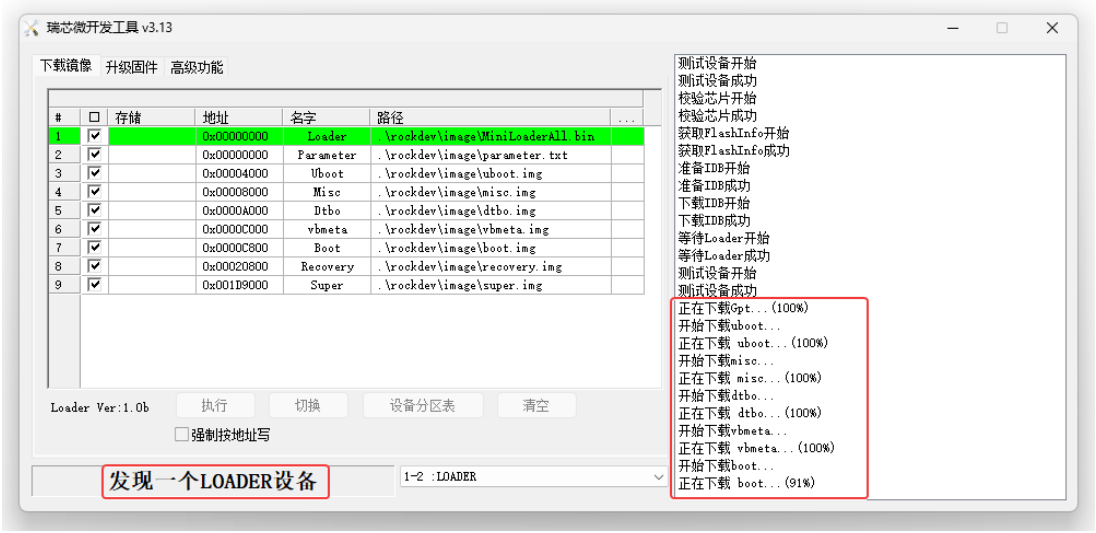
- 然后给主板接上 Type-C 接口的电源，并上电



- 等待软件提示发现一个 LOADER 设备（如下图所示），然后就可以松开按键



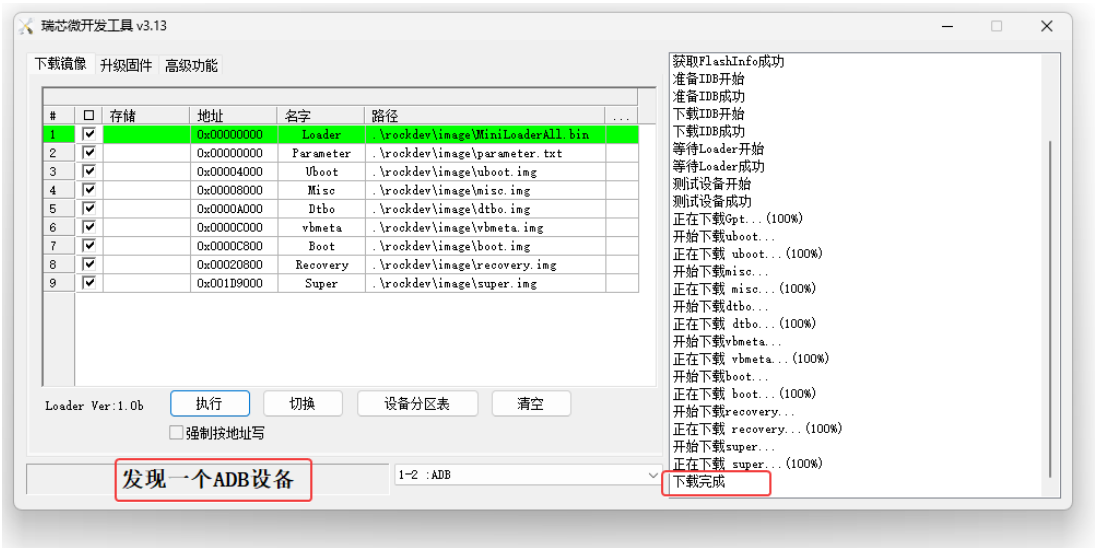
- 如果不成功，重复如上步骤
- 然后点击执行开始固件烧录



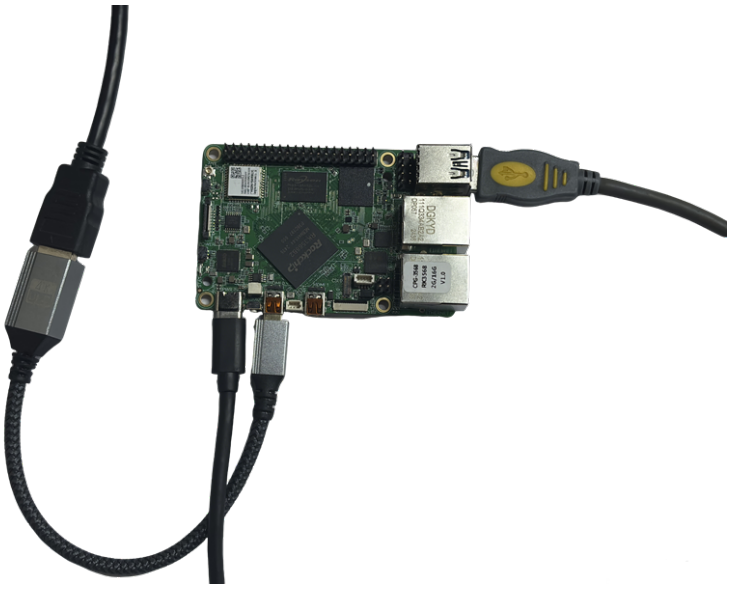
- 镜像烧录成功后，等待 10 秒左右，提示发现一个 ADB 设备，如下图:

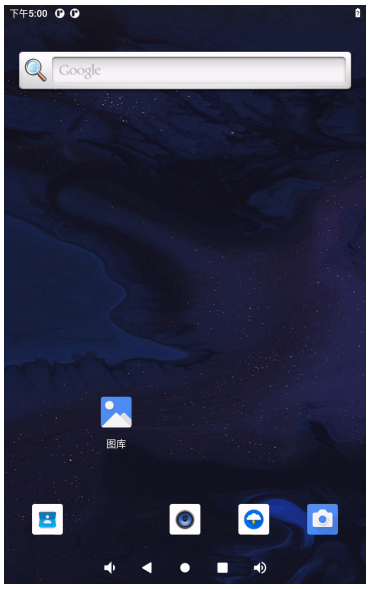
注意

请勿使用电脑 USB 口作为电源供电，否则无法发现 ADB 设备



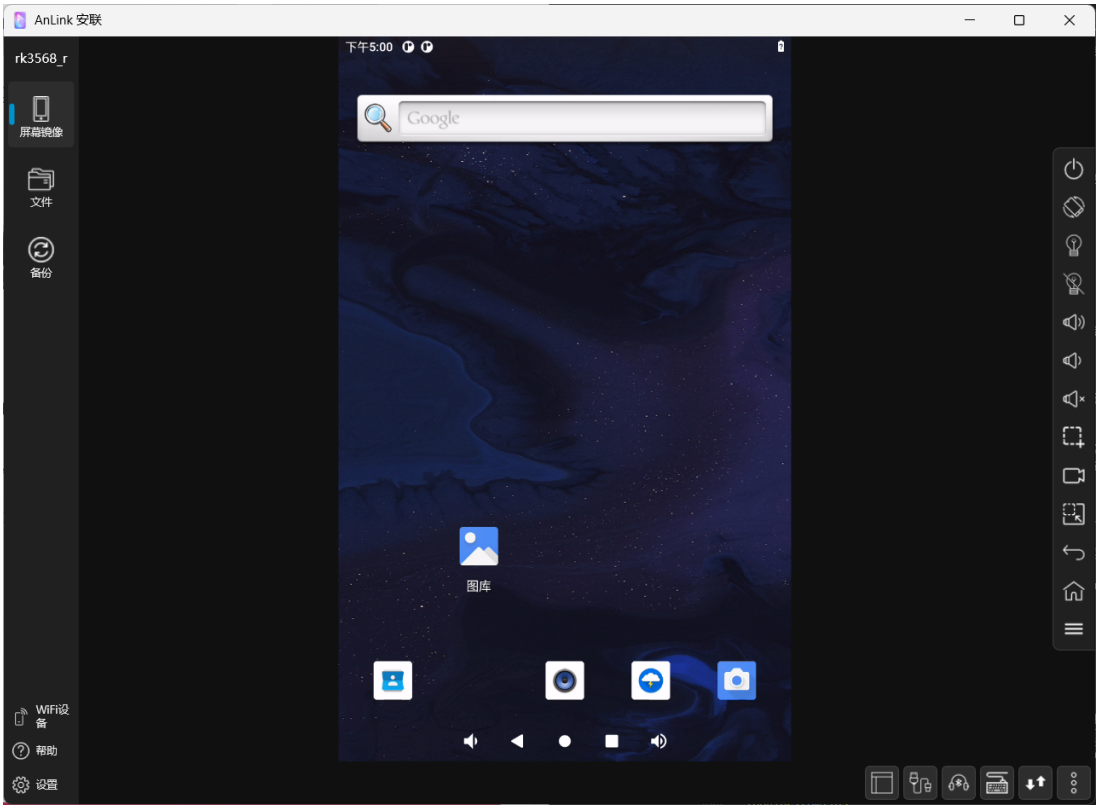
- 使用Micro HDMI 转 HDMI 线和HDMI 线连接显示器，即可显示安卓系统界面





- 也可使用投屏软件 AnLinkSetup.exe，连接 adb 后自动显示安卓系统界面

投屏软件 AnLinkSetup.exe 路径：**可乐派网盘->CPG-3568-> 工具软件**



2.4 启动可乐派主板

- 主板有 Micro HDMI 接口，可以通过 Micro HDMI 转 HDMI 线把主板连接到电视或者 HDMI 显示器。如果有购买我司 8 寸 MIPI 屏幕 (型号 xmz080cs6)，也可以使用 MIPI 屏幕来显示主板的系统界面。如烧录章节所述2.3.3，也可使用投屏软件 anlinksetup.exe 来显示系统界面
- 接上 USB 鼠标和键盘，用于控制可乐派主板

- 可乐派主板有以太网口，可以插入网线用来上网
- 连接一个 5V-5A 的 USB Type-C 接口的高品质的电源适配器

注意

切记不要插入电压输出大于 5V 的电源适配器，会烧坏主板。系统上电启动过程中很多不稳定的现象基本都跟供电有关，如果启动过程中发现有不断重启的现象，或者无法识别 adb 设备，请更换下电源或者双公头 USB 数据线再试下。请不要接到电脑的 USB 接口来给主板供电

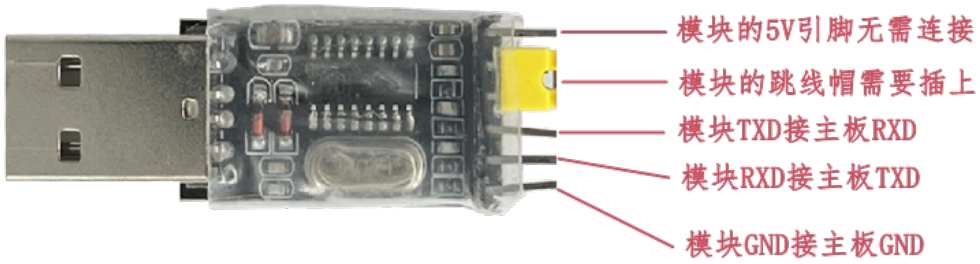
- 如果一切正常，此时 HDMI 显示器或者 MIPI 屏幕就能看到系统的启动画面了
- 如果想通过调试串口查看系统的输出信息，请使用串口线将主板连接到电脑。串口的连接方法请参看下面的2.5 一节

在嵌入式设备或相关系统启动过程中，HDMI 接口和 MIPI 接口分别承担着连接不同类型屏幕的关键任务。HDMI 显示器通用性强，广泛应用于各类显示场景；MIPI 屏幕则在一些对空间、功耗等有特殊要求的设备中使用。当系统与这些屏幕的连接、驱动以及相关配置均无误时，屏幕会准确呈现出系统启动画面，这一画面不仅是系统正常启动的直观体现，也为后续的操作和调试提供了基础

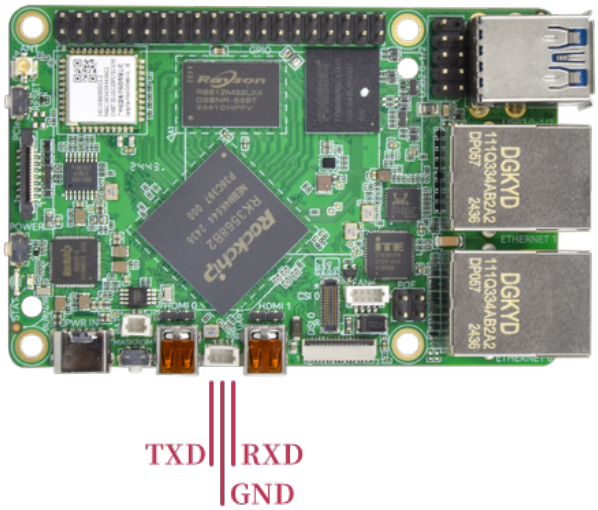
2.5 调试串口的使用方法

2.5.1 调试串口的连接说明

- 首先需要准备一个 3.3V 的 USB 转 TTL 模块，然后将 USB 转 TTL 模块的 USB 接口一端插入到电脑的 USB 接口中



- 主板的调试串口 GND、RXD 和 TXD 引脚的对应关系如下图所示



- USB 转 TTL 模块 GND、TXD 和 RXD 引脚需要通过SH1.0 转杜邦线 2.54 3P 连接到主板的调试串口上

  1. USB 转 TTL 模块的 GND 接到主板的 GND 上
  2. USB 转 TTL 模块的 RXD 接到主板的 TXD 上
  3. USB 转 TTL 模块的 TXD 接到主板的 RXD 上

- USB 转 TTL 模块连接电脑和 CokePi 主板的示意图如下所示

串口的 TX 和 RX 是需要交叉连接的，也可以把串口的 TX 和 RX 先随便接上，如果测试没有输出再交换下，同时要确保串口波特率是 **150 0000**

2.5.2 Ubuntu 平台调试串口的使用方法

Linux 下可以使用的串口调试软件有 minicom、putty 等，下面介绍 putty 的使用方法

- 首先将 USB 转 TTL 模块插入 Ubuntu 电脑的 USB 接口，如果 USB 转 TTL 模块连接识别正常，在 Ubuntu PC 的/dev 下就可以看到对应的设备节点名，记住这个节点名，后面设置串口软件时会用到

```
teamhd@teamhd:~$ ls /dev/ttyUSB*
/dev/ttyUSB0
```

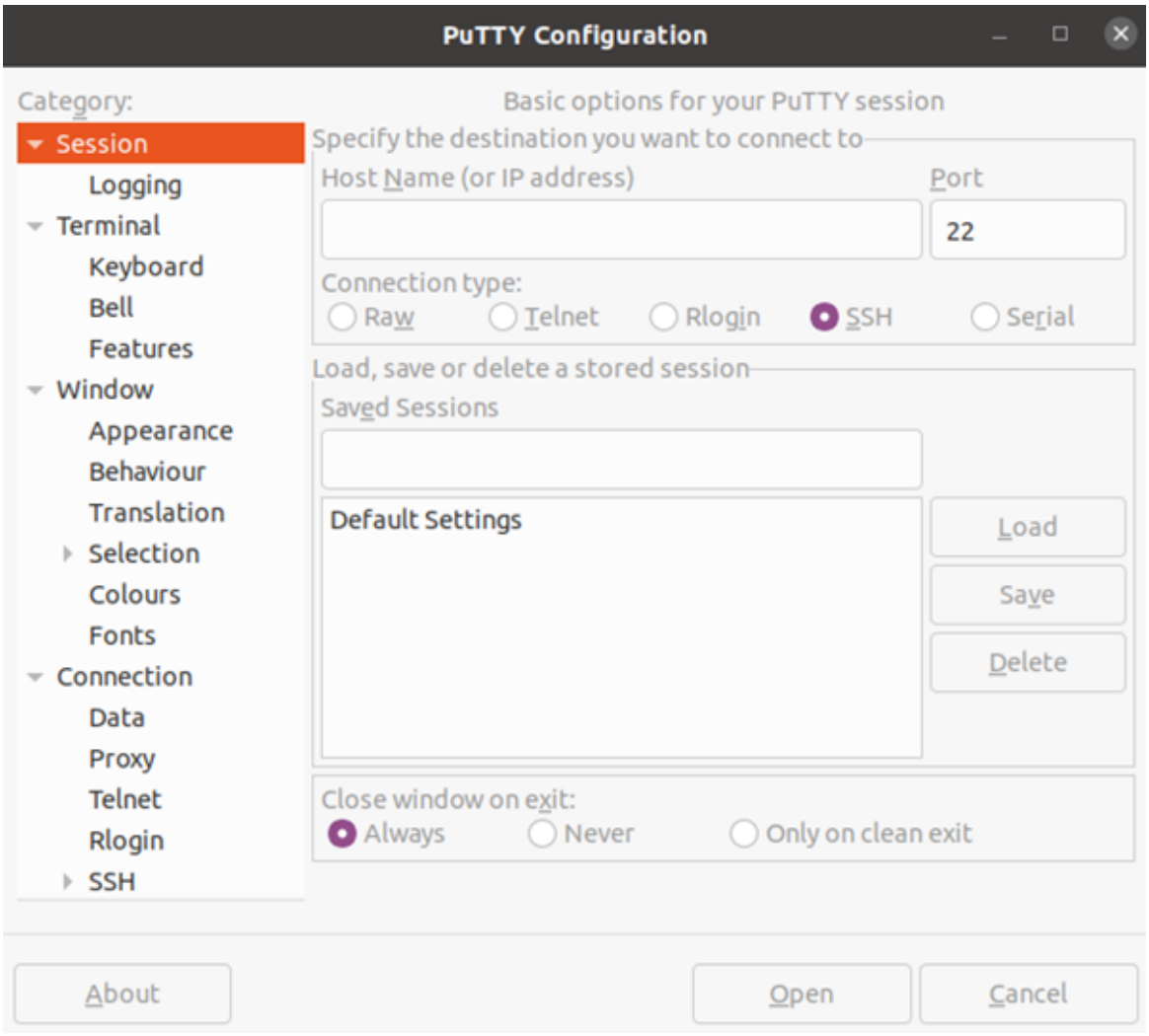
- 然后使用如下的命令在 Ubuntu PC 上安装 putty

```
teamhd@teamhd:~$ sudo apt-get update -y
teamhd@teamhd:~$ sudo apt-get install -y putty
```

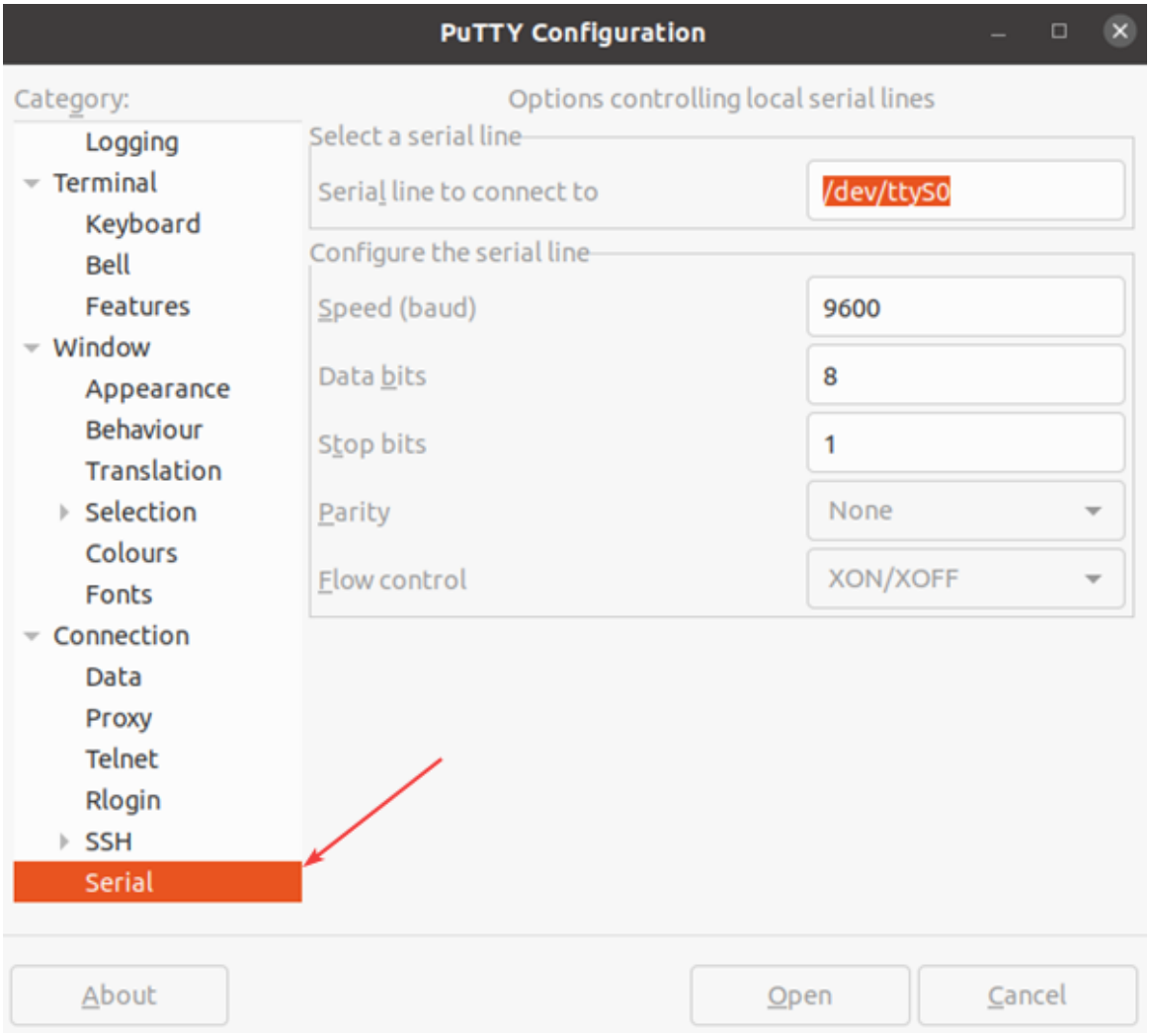
- 然后运行 putty，需加 sudo 权限

```
teamhd@teamhd:~$ sudo putty
```

- 执行 putty 命令后会弹出下面的界面

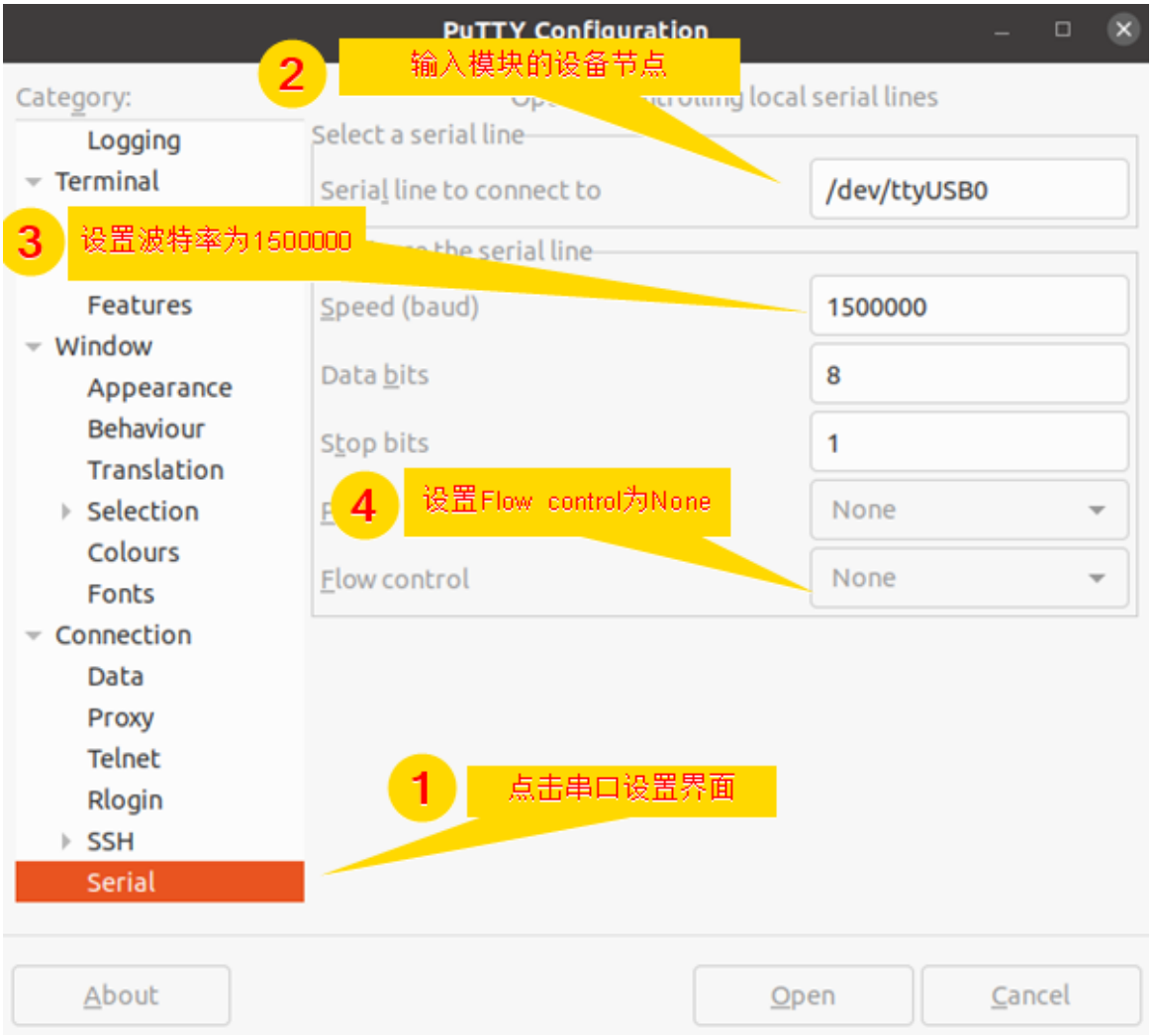

PuTTY Configuration
Category:
Session
Logging
Terminal
Keyboard
Bell
Features
Window
Appearance
Behaviour
Translation
Selection
Colours
Fonts
Connection
Data
Proxy
Telnet
Rlogin
SSH
Basic options for your PuTTY session
Specify the destination you want to connect to
Host Name (or IP address)
Port
22
Connection type:
Raw
Telnet
Rlogin
SSH
Serial
Load, save or delete a stored session
Saved Sessions
Default Settings
Load
Save
Delete
Close window on exit:
Always
Never
Only on clean exit
About
Open
Cancel

- 首先选择串口的设置界面



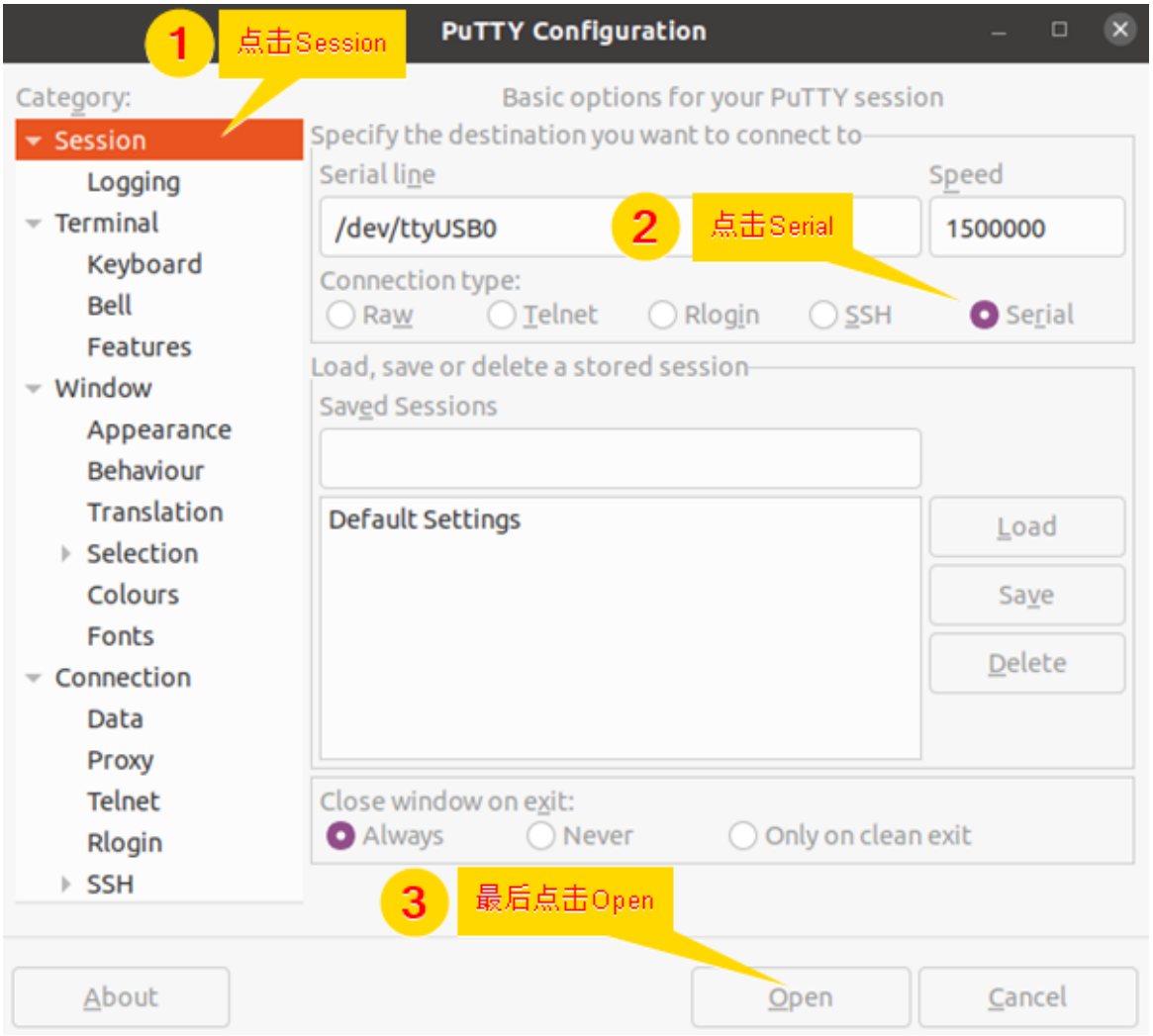
- 然后设置串口的参数

1. 设置 Serial line to connect to 为/dev/ttyUSB0（修改为对应的节点名，一般为 /dev/t-tyUSB0）
2. 设置 Speed(baud) 为 1500000（串口的波特率）
3. 设置 Flow control 为 None

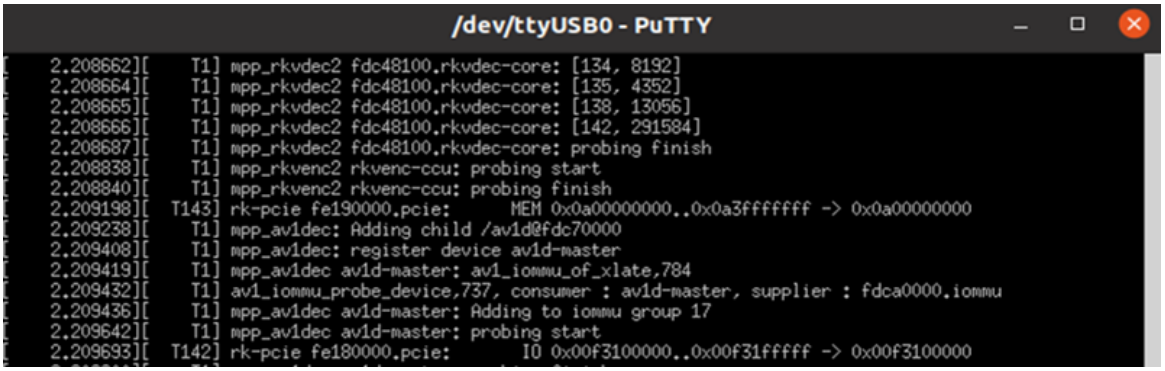


- 在串口的设置界面设置完后，再回到 Session 界面

  1. 首先选择 Connection type 为 Serial
  2. 然后点击 Open 按钮连接串口



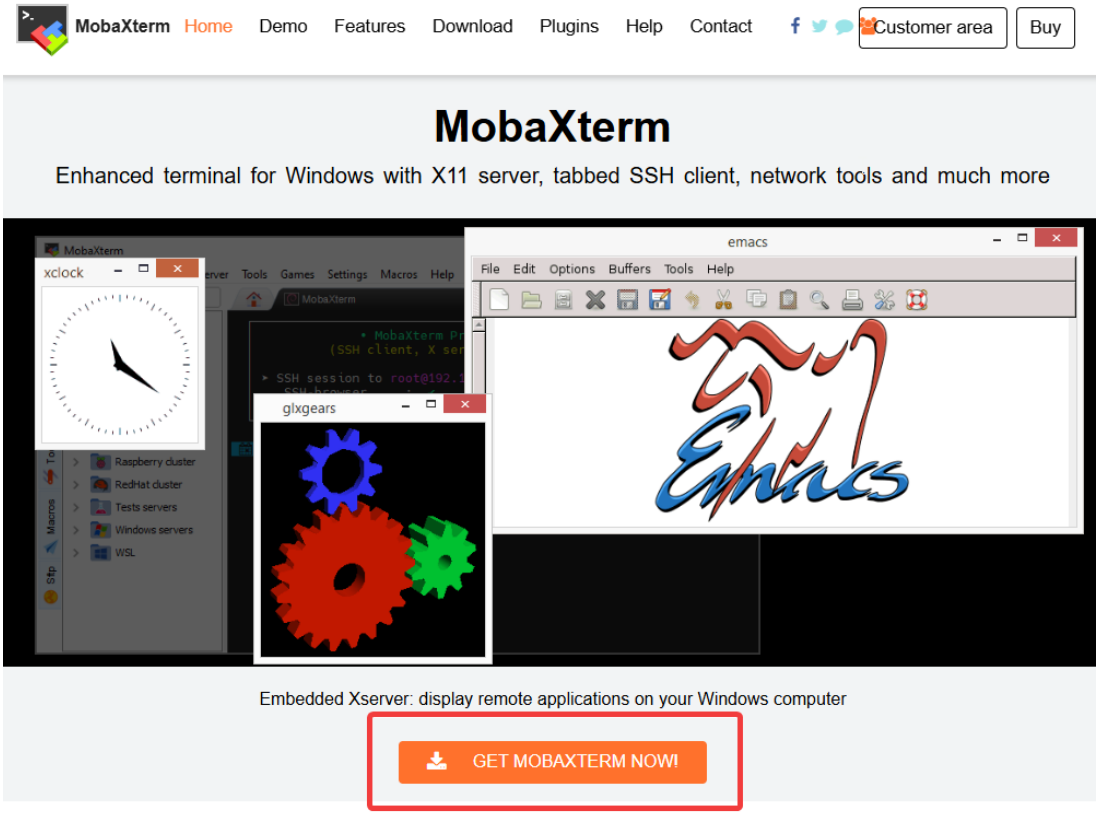
- 启动主板后，就能从打开的串口终端中看到系统输出的 Log 信息了



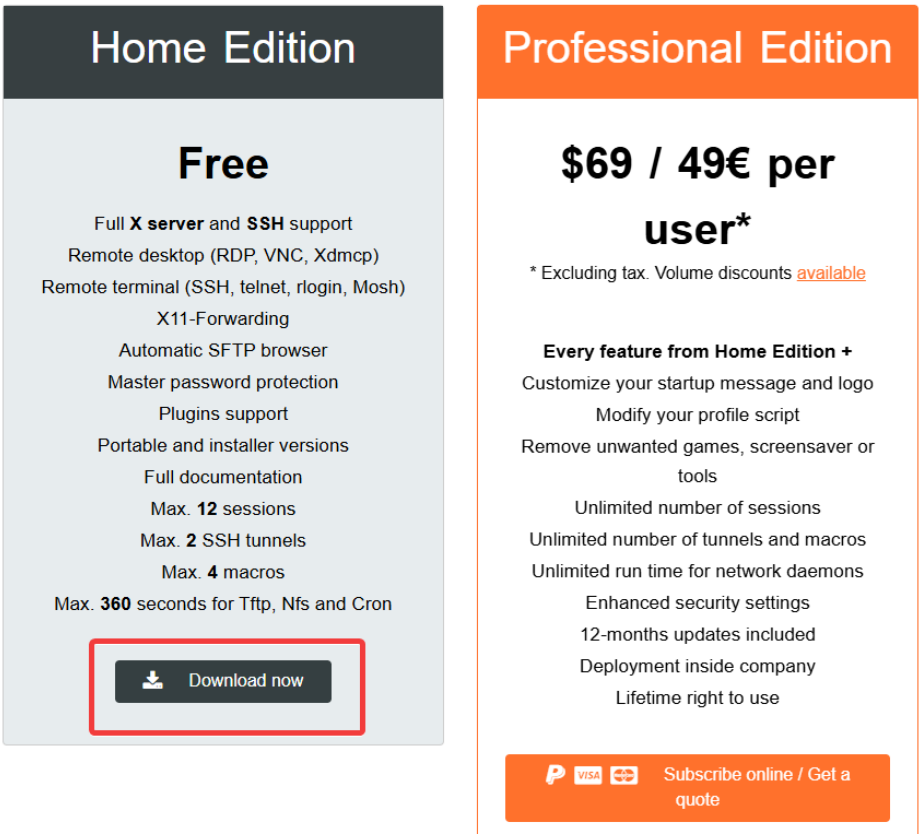
2.5.3 Windows 平台调试串口的使用方法

Windows 下可以使用的串口调试软件有:MobaXterm、SecureCRT 等，下面演示 MobaXterm 的使用方法，这款软件有免费版本，无需购买序列号即可使用

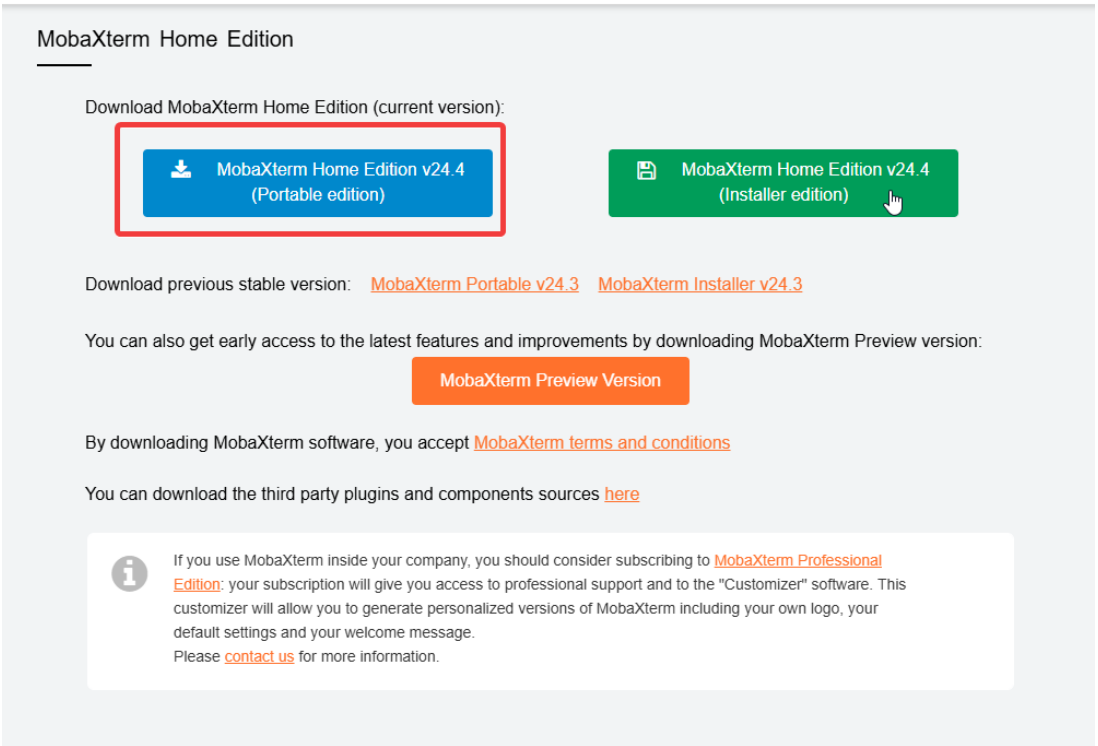
- 下载 MobaXterm 网址 https://mobaxterm.mobatek.net
- 进入 MobaXterm 下载网页后点击 GET XOBATERM NOW!



- 然后选择下载 Home 版本

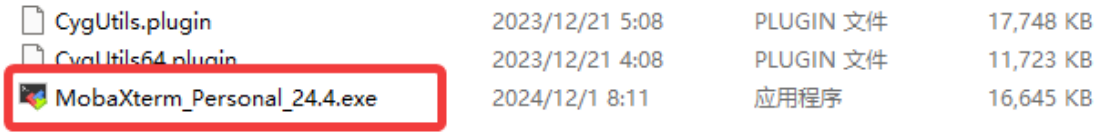


- 然后选择 Portable 便携式版本，下载完后无需安装，直接打开就可以使用



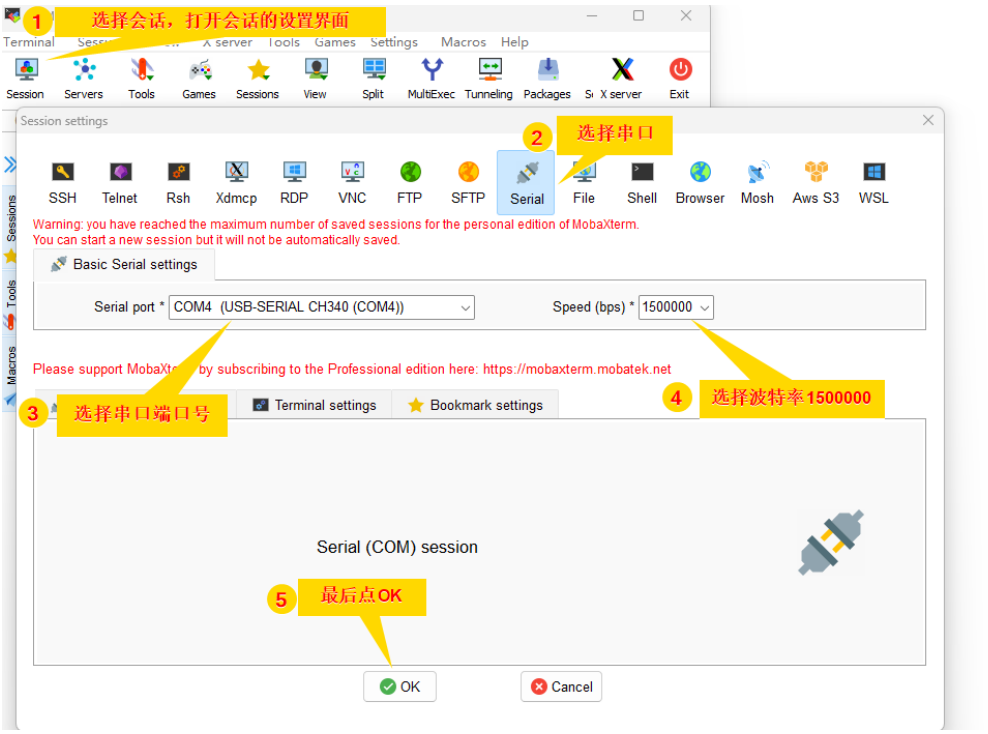
Changelog

- 下载完后使用解压缩软件解压下载的压缩包，即可得到 MobaXterm 的可执软件，然后双击打开

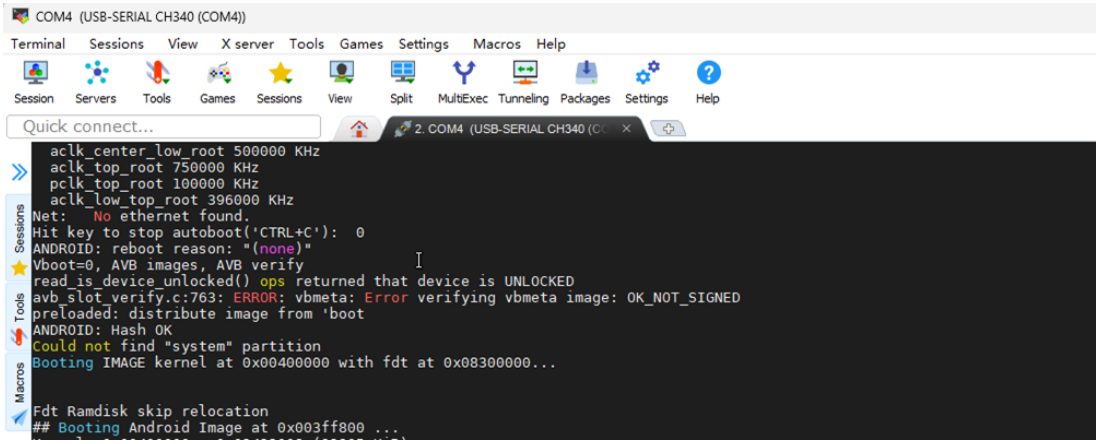


- 打开软件后，设置串口连接的步骤如下

  1. 打开会话的设置界面
  2. 选择串口类型
  3. 选择串口的端口号（根据实际的情况选择对应的端口号），如果看不到端口号，请使用 360 驱动大师扫描安装 USB 转 TTL 串口芯片的驱动
  4. 选择串口的波特率为 1500000
  5. 最后点击”OK” 按钮完成设置



- 点击”OK” 按钮后会进入下面的界面，此时启动主板就能看到串口的输出信息了



2.6 使用可乐派主板 40-Pin 接口中的 5v 引脚供电说明

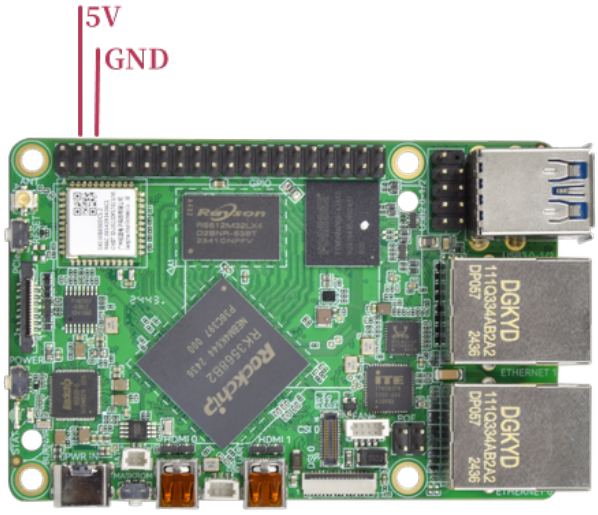
提示

我们推荐使用 5V-5A 的 Type-C 接口的电源线插到主板的 Type-C 电源接口来供电。如果需要使用 40-Pin 接口中的 5V 引脚来给主板供电，请确保使用的电源线和电源适配器能满足主板的供电需求。如果有使用不稳定的情况，请换回 Type-C 电源供电

- 首先需要准备一根下图所示的电源线 (淘宝可以买到，请自行购买)

- 使用 40-Pin 接口中的 5V 引脚来给主板供电，电源线的接法如下所示

1. 上图所示的电源线 USB A 口需要插到 5V-5A 的电源适配器接头上
   请不要插到电脑的 USB 接口来供电
2. 红色的杜邦线需要插到主板 40-Pin 插针的 5V 引脚上
3. 黑色的杜邦线需要插到主板 40-Pin 插针的 GND 引脚上
4. 40-Pin 插针 5V 引脚和 GND 引脚在主板中的位置如下图所示，切记不要接反了


5V
GND

3 Ubuntu LXQt 桌面系统使用说明

本章内容是基于 Ubuntu LXQt 桌面版本镜像编写的

3.1 本手册 Linux 命令格式说明

本手册中所有需要在 Linux 系统中输入的命令都会使用下面的方框框起来

```
$
```

3.2 Linux 系统登录说明

3.2.1 系统默认登录账号和密码

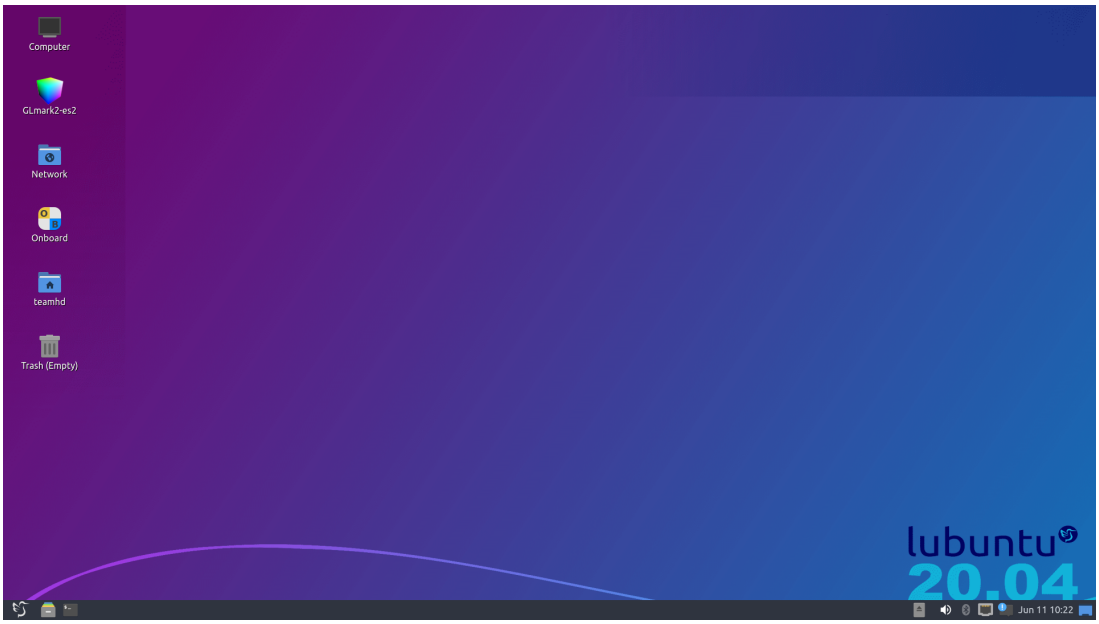
账号	密码
root	root
teamhd	teamhd

提示

在输入密码时，屏幕不会显示输入的密码具体内容，这并非设备存在故障。输入完成后直接按下回车键即可。这种设计主要是出于安全考量，防止密码在输入过程中被他人窥视，从而保障用户信息和设备系统的安全。用户只需按照常规操作输入密码，完成输入动作后按下回车键，系统便会对输入的密码进行验证，以继续后续的操作流程

3.2.2 Linux 系统终端登录的方法

- Linux 系统默认就是自动登录终端的，默认登录的用户名是teamhd，登录后的界面如下图所示:



- ssh 允许 root 用户登录

```
sudo sed -i 's/^#PermitRootLogin prohibit-password$/PermitRootLogin yes/' /etc/ssh\
/sshd_config


sudo systemctl restart ssh
```

- 禁止自动登录终端

```
sudo vim /etc/lightdm/lightdm.conf
```

添加如下语句

```
    [Seat:*]
    autologin-user=
    autologin-user-timeout=0
```

```
    sudo reboot
```

- 自动登录，并设置需要自动登录的用户名

将 autologin-user= 后面的用户名改为需要自动登录的用户名, 例如：以 root 登录

```
    [Seat:*]
    autologin-user=root
    autologin-user-timeout=0
```

- 禁用桌面

```
sudo systemctl disable lightdm.service

sudo reboot
```

- 启用桌面

```
sudo systemctl start lightdm.service

sudo systemctl enable lightdm.service

sudo dpkg-reconfigure lightdm
```

3.2.3 Linux 桌面版系统自动登录说明

- 桌面版系统启动后会使用普通用户teamhd自动登录进入桌面，无需输入密码
- 使用 root 用户登录的方法

编辑 lightdm 的配置文件：

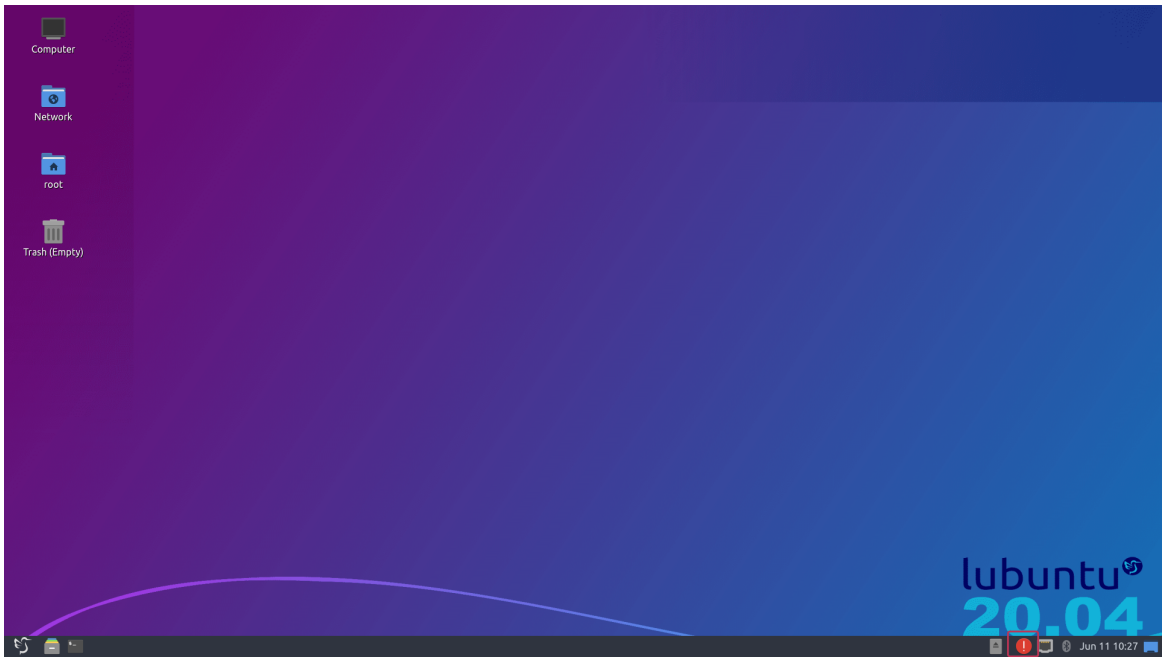
```
sudo vim /etc/lightdm/lightdm.conf
```

添加以下行：

```
    [Seat:*]
    autologin-user=root
    autologin-user-timeout=0
```

重启主板：

```
sudo reboot
```

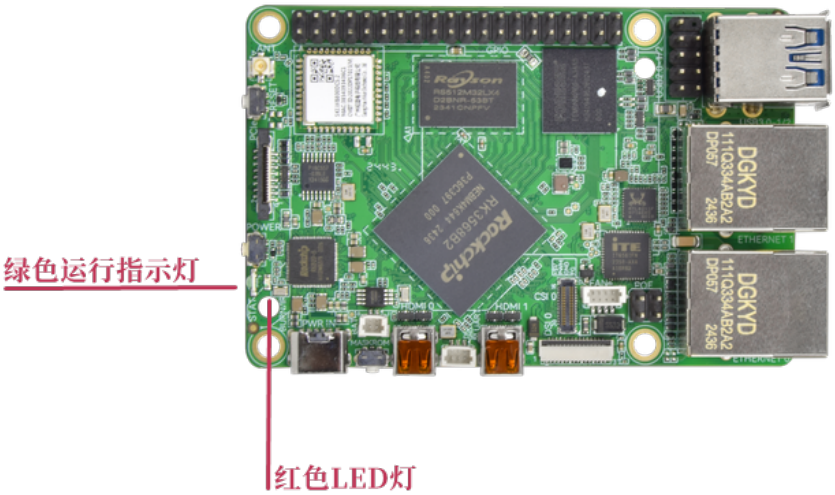


提示

如果使用 root 用户登录桌面系统，是无法使用右上角的 pulseaudio 来管理音频设备的。这并不是一个 bug，因为 pulseaudio 默认不允许在 root 用户下运行

3.3 板载 LED 灯测试说明

- 主板上有两个 LED 灯，一个绿灯，一个红灯，所在位置如下图所示：



- 绿色的 LED 灯在内核启动后会一直闪烁，这是由软件控制的

  设置绿灯亮灭和闪烁的方法如下所示

提示

下面的操作请在 root 用户下进行

a. 首先进入绿灯的设置目录

```
cd /sys/devices/platform/leds/leds/work
```

b. 设置绿灯停止闪烁的命令如下

```
echo none > trigger
```

c. 继续执行如下命令，设置绿灯常亮

```
echo 255 > /sys/devices/platform/leds/leds/work/brightness
```

d. 恢复绿灯闪烁的命令如下

```
echo heartbeat > trigger
```

- 红色的 LED 灯默认常灭，可以通过 GPIO 控制

```
#使能引脚 GPIO3_D4
echo 124 > /sys/class/gpio/export

# 设置引脚为输出模式
echo out > /sys/class/gpio/gpio124/direction

# 设置引脚为高电平
echo 1 > /sys/class/gpio/gpio124/value

# 复位引脚
echo 124 > /sys/class/gpio/unexport
```

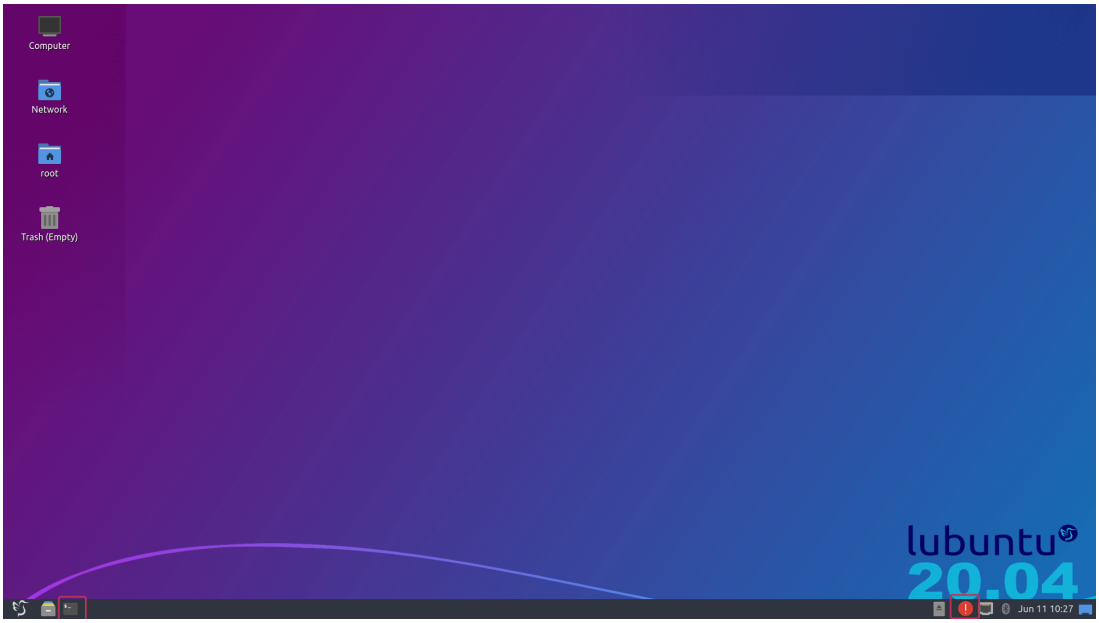
3.4 网络连接测试

3.4.1 以太网口测试

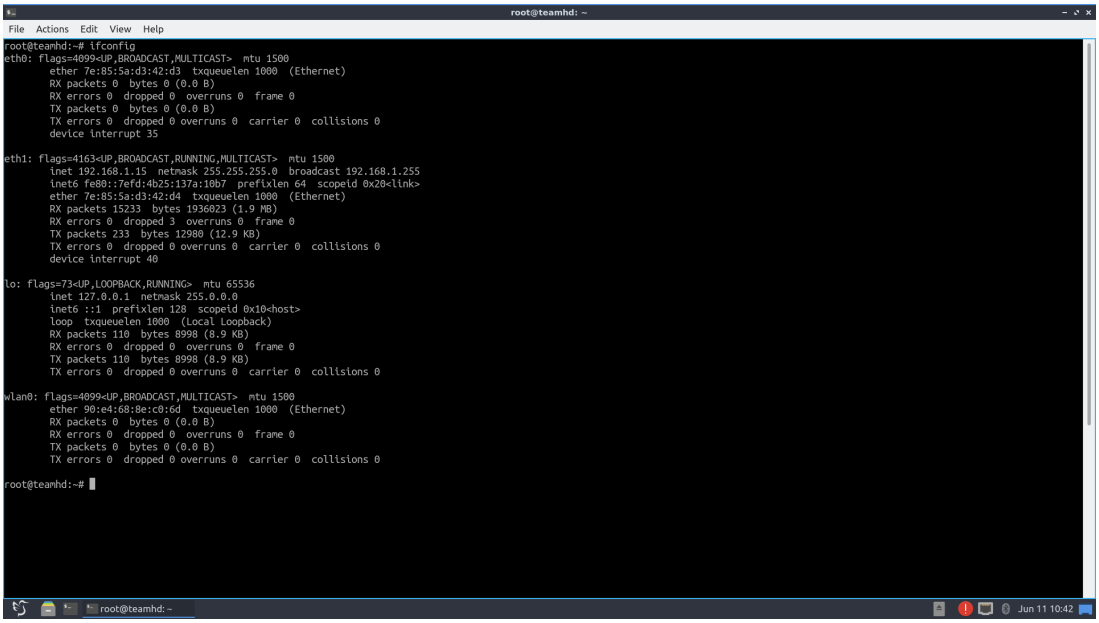
1. 首先将网线的一端插入可乐派主板的以太网接口，网线的另一端接入路由器，并确保网络是畅通的

2. 系统启动后会通过 DHCP 自动给以太网卡分配 IP 地址，不需要其他任何配置

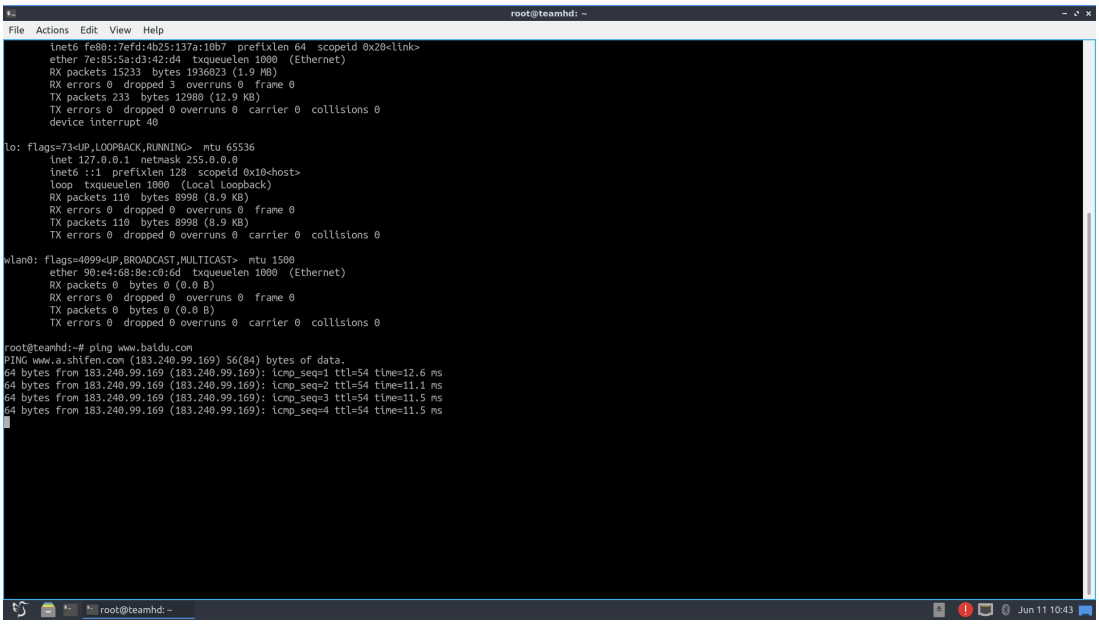
3. 鼠标点击**左下角的第三个图标**，打开命令行



4. 在命令行中输入 ifconfig 查看 IP 地址如下所示

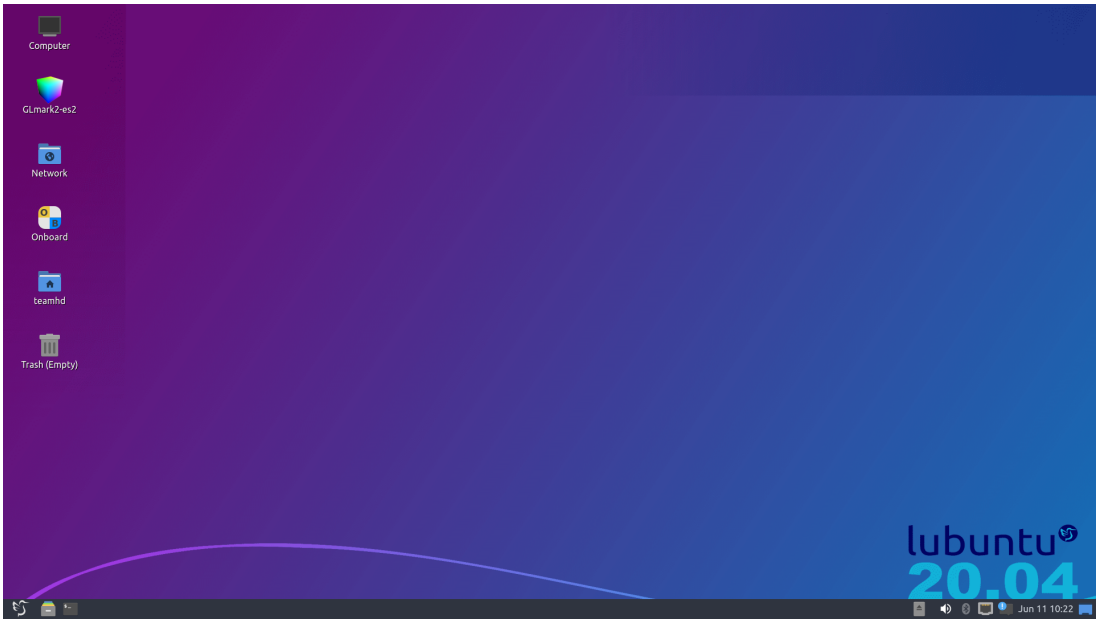


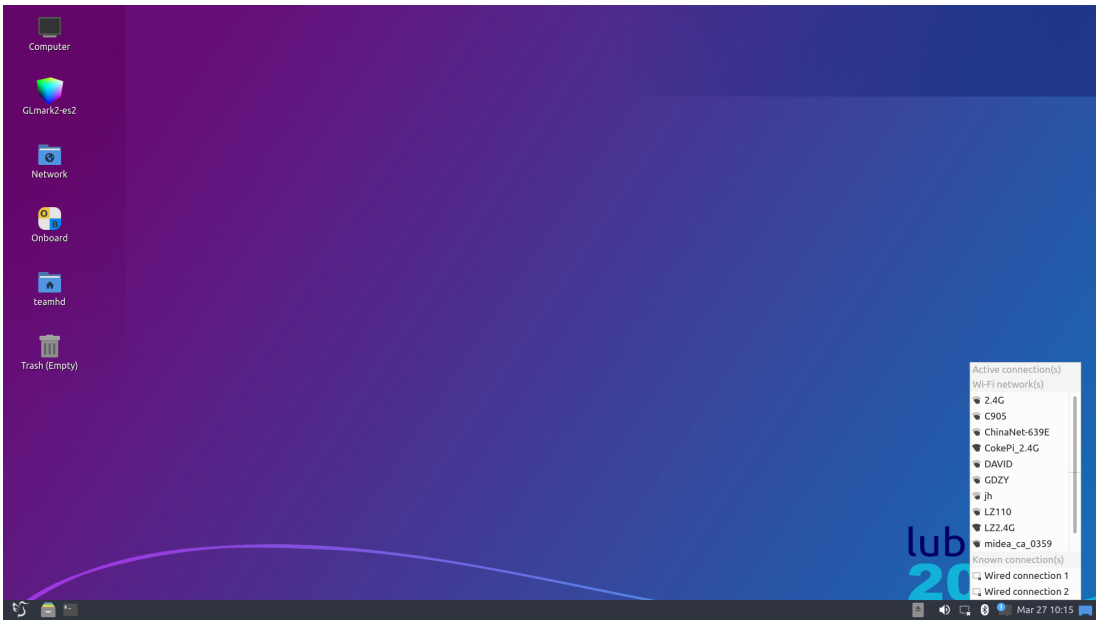
5. 测试网络连通性的命令如下，ping 命令可以通过 Ctrl+C 快捷键来中断运行



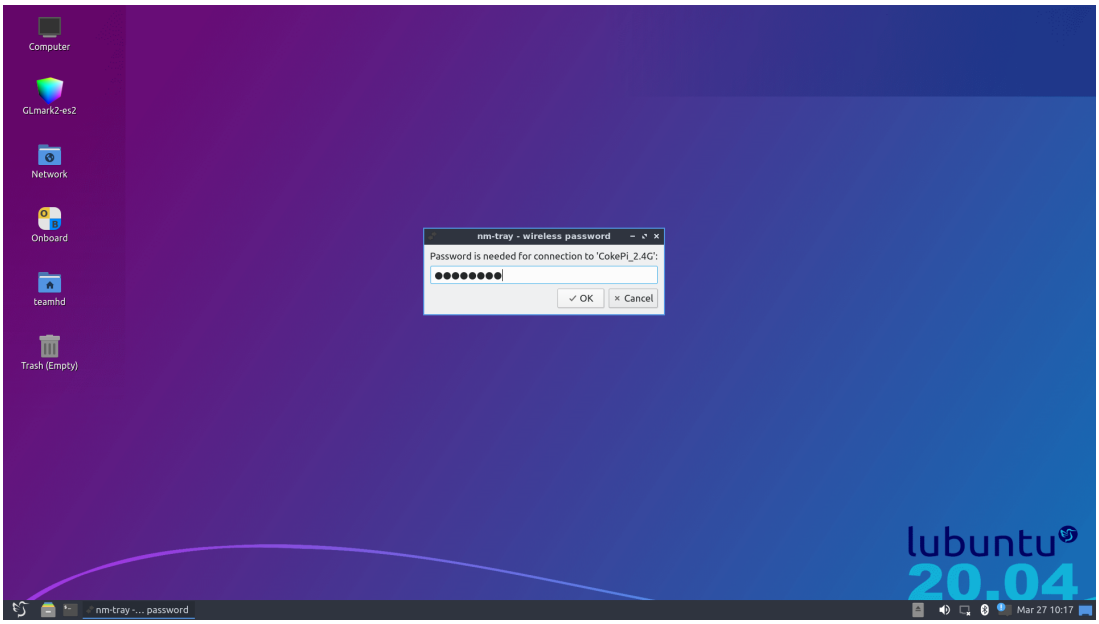
3.4.2 WiFi 连接测试

1. 测试 WiFi 连接请拔掉网线，并接上WiFi 天线，然后点击**右下角的网络图标**，选择需要连接的 WiFi 网络

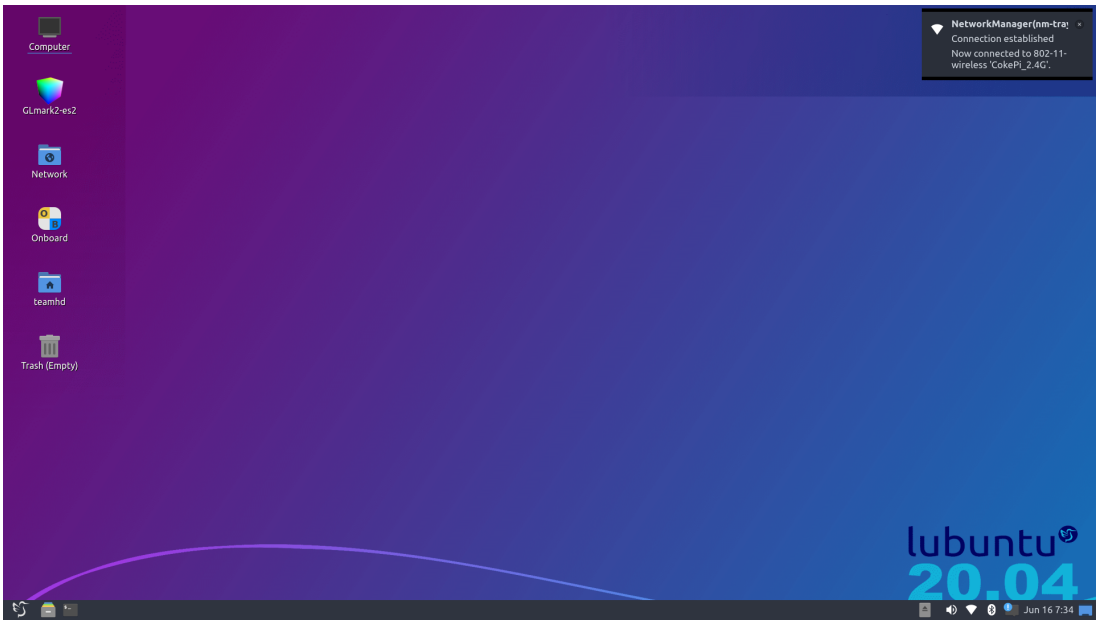




2. 连接键盘，输入 WiFi 密码，点击 Connect 连接

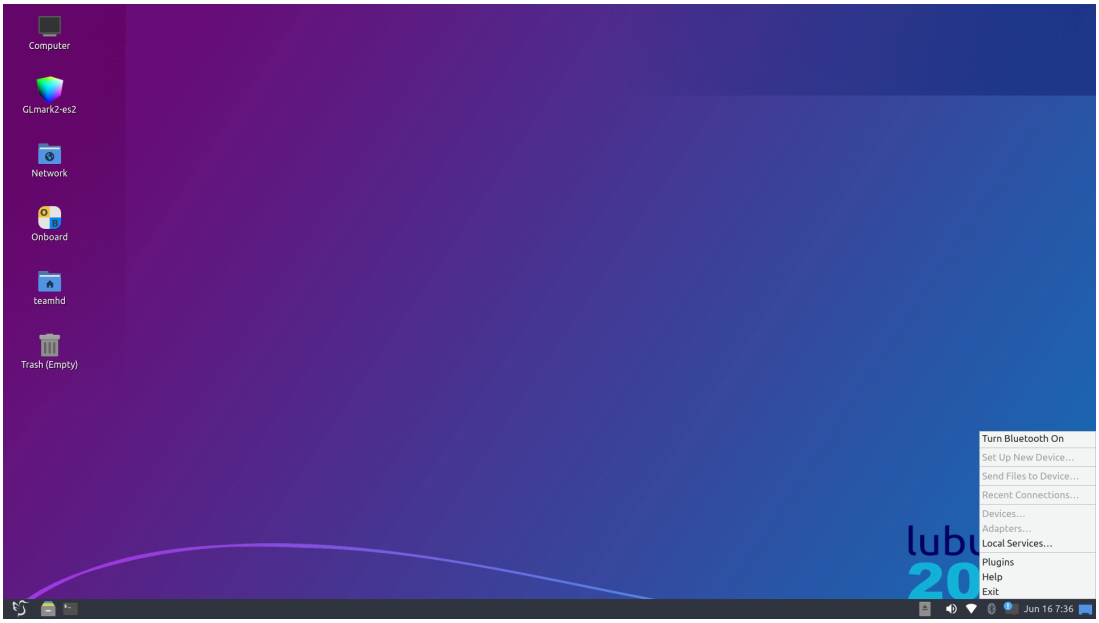


3. 连接成功后，右上角的网络图标会显示连接成功的 WiFi 网络

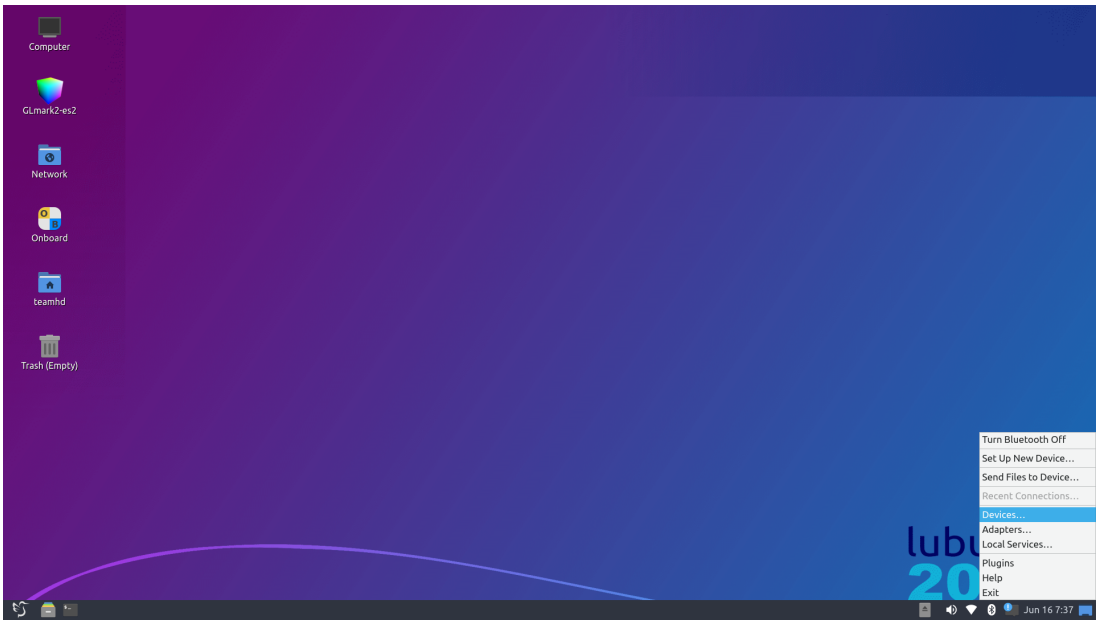


3.5 蓝牙连接测试

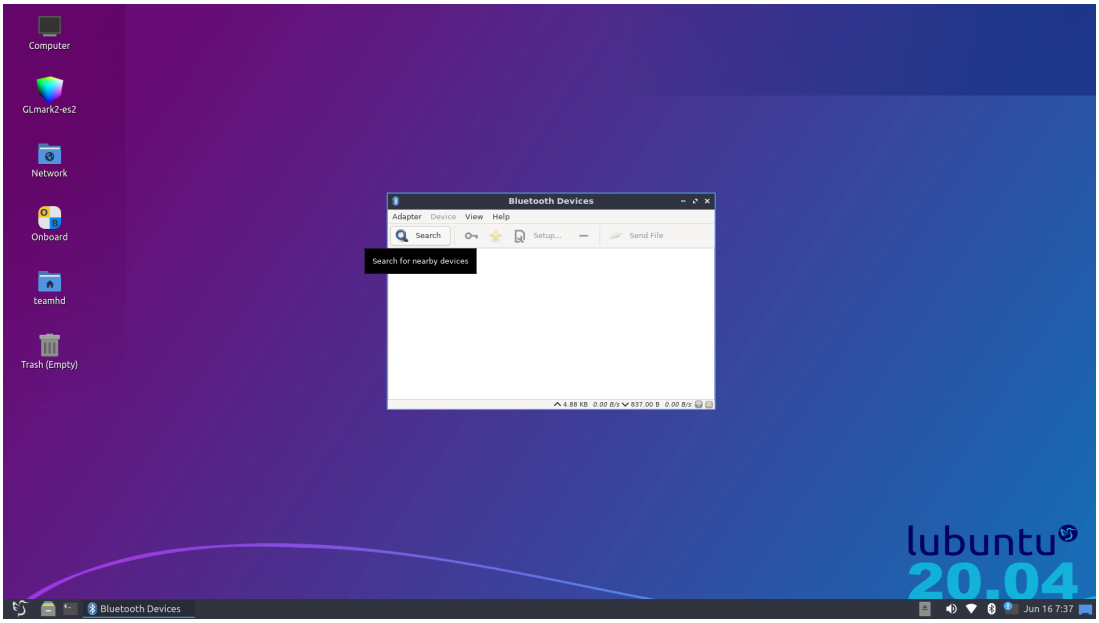
1. 测试蓝牙连接请拔掉网线，并接上WiFi 天线(WiFi 和蓝牙共用一根天线)，然后点击右上角的蓝牙图标，点击 **Turn Bluetooth On** 打开蓝牙



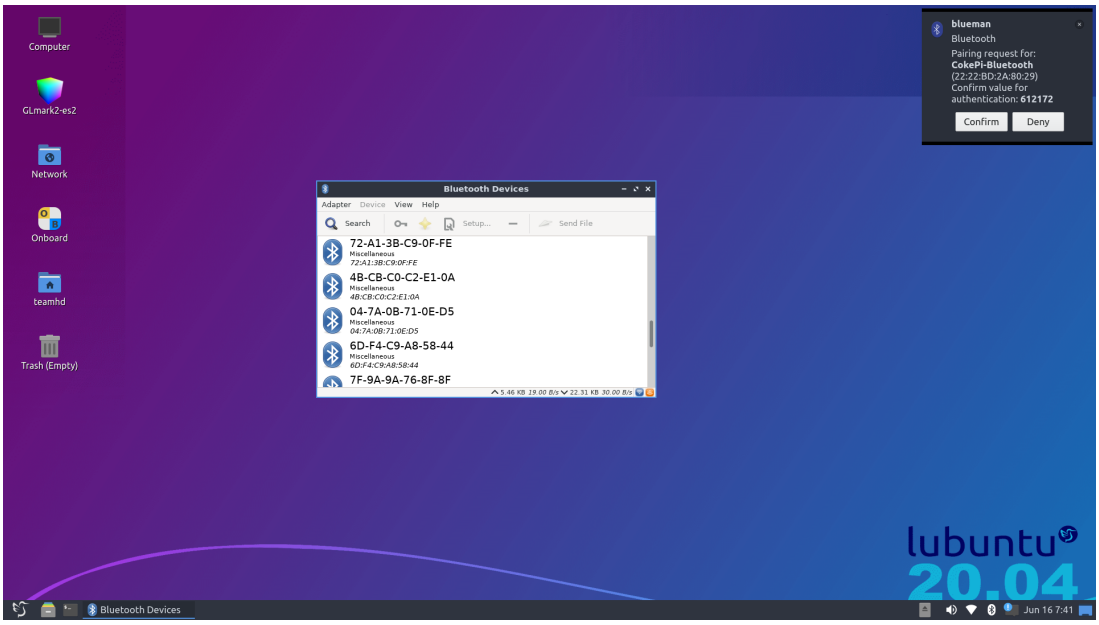
2. 再次点击**右下角的蓝牙图标**，点击 **Devices**，进入蓝牙设备连接界面



3. 点击 **Search**，开始扫描蓝牙设备

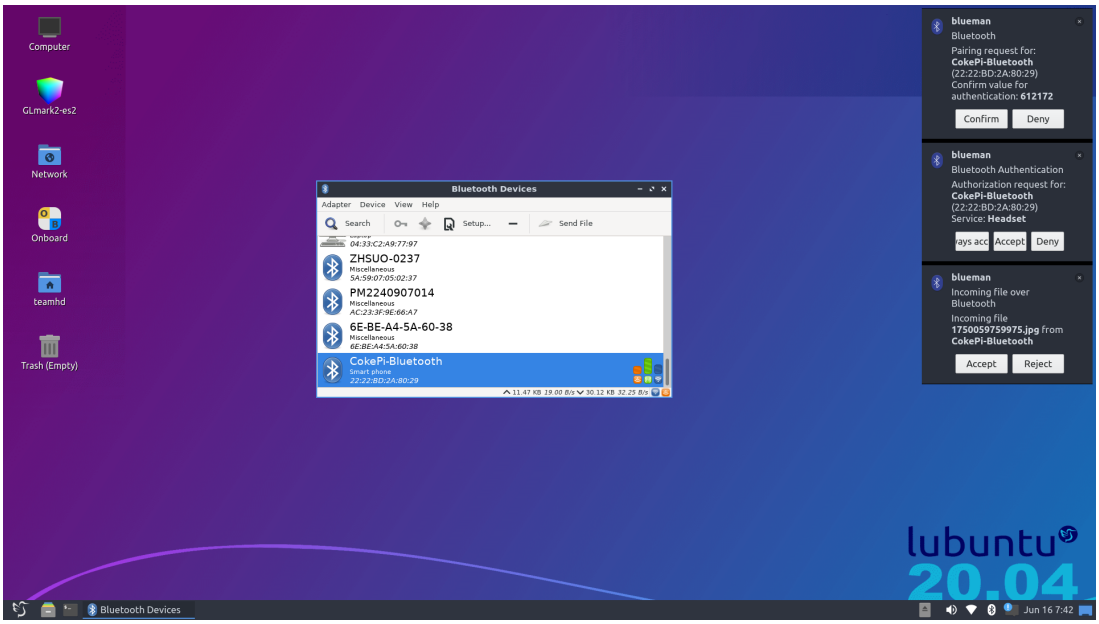


4. 选中需要连接的蓝牙设备，鼠标右键点击 **Connect**，开始连接



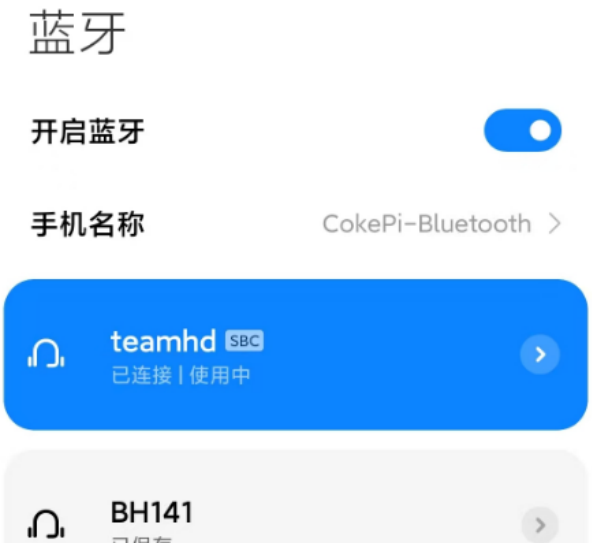
5. 手机端点击**配对**

6. 鼠标移动到右上角弹出的蓝牙窗口，点击 **Confirm**，确认连接

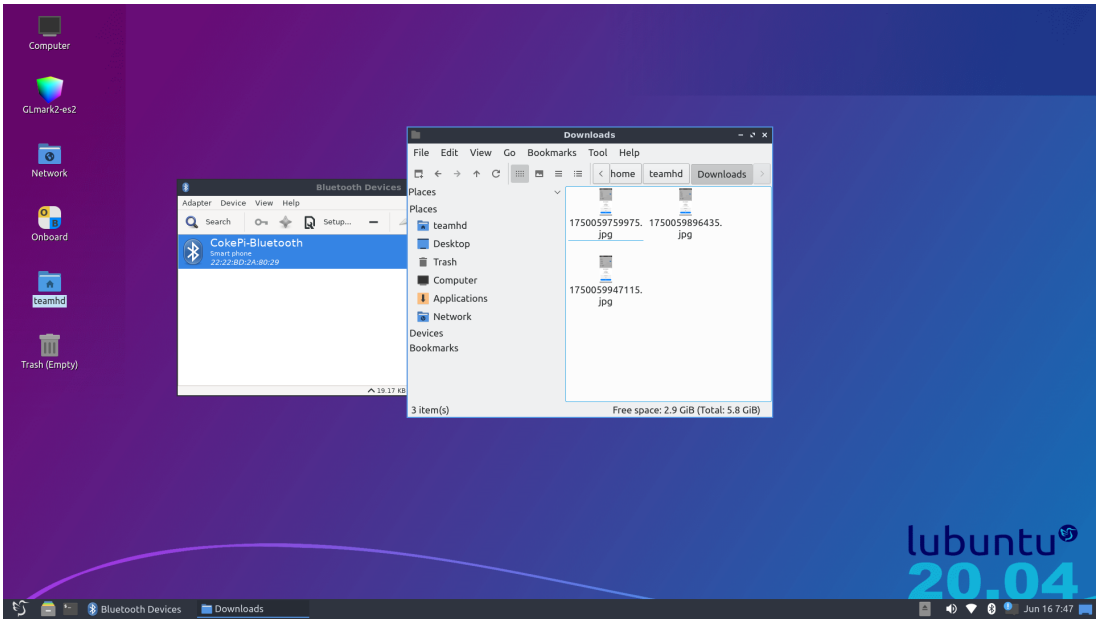


7. 连接成功后，再次点击 **Always accept** 始终接受其连接请求

8. 手机端提示已连接代表连接成功，可以进行数据传输



9. 主板接收文件成功，在/home/teamhd/Downloads目录下可以看到接收到的文件



4 Linux SDK 使用说明

4.1 SDK 简介

瑞芯微通用 Linux SDK 工程，适用于以 Rockchip 处理器为主芯片的可乐派系列板卡。整个 SDK 工程，目录包含有 buildroot、debian、app、kernel、u-boot、device、external 等目录。每个目录或其子目录会对应一个 git 工程，提交需要在各自的目录下进行。

- app：存放上层应用 app，主要是 qcamera/qfm/qplayer/settings 等一些应用程序，在 buildroot 构建时会使用到这些程序
- buildroot：基于 buildroot 开发的根文件系统
- debian：Debian 根文件系统构建脚本
- device/rockchip：存放各芯片板级配置和 Parameter 文件，以及一些构建与打包固件的脚本和预备文件
- external：存放第三方相关仓库, 包括音频、视频、网络、recovery 等，在 buildroot 构建时将会用到
- kernel：存放 kernel 源码
- prebuilts：存放交叉编译工具链
- rkbin：存放 Rockchip 相关的 Binary 和工具
- rockdev：存放编译输出固件
- tools：存放 Linux 和 Windows 操作系统环境下常用工具
- u-boot：存放进行开发的 uboot 代码

4.2 SDK 开发环境搭建

CokePi-SDK 是基于 Ubuntu 22.04 系统开发测试的，为了不必要的麻烦，我们推荐用户使用 Ubuntu22.04。磁盘空间建议大于 300GB，内存建议大于 4GB。

4.2.1 安装 SDK 依赖的软件包

```
# 安装 SDK 构建所需要的软件包
# 整体复制下面内容到终端中安装

sudo apt install git ssh make gcc libssl-dev liblz4-tool u-boot-tools curl expect \
g++ patchelf chrpath gawk texinfo chrpath diffstat binfmt-support \
qemu-user-static live-build bison flex fakeroot cmake gcc-multilib \
g++-multilib unzip device-tree-compiler python3-pip libncurses5-dev \
python3-pyelftools dpkg-dev
```

4.2.2 安装 repo

```
mkdir ~/bin

curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo

chmod a+x ~/bin/repo

echo PATH=~/bin:$PATH >> ~/.bashrc

source ~/.bashrc
```

- 执行完上面的命令后我们来验证 repo 是否安装成功能正常运行

```
repo --version

# 返回以下信息
# 返回的信息根据Ubuntu版本的不同略有差异
<repo not installed>
repo launcher version 2.50
      (from /home/teamhd/bin/repo)
git 2.34.1
Python 3.10.12 (main, Feb 4 2025, 14:57:36) [GCC 11.4.0]
OS Linux 6.8.0-52-generic (#53~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Wed Jan 15
    19:18:46 UTC 2)
CPU x86_64 (x86_64)
Bug reports: https://issues.gerritcodereview.com/issues/new?component=1370071
```

4.2.3 安装 python2.7

```
cd ~

wget https://www.python.org/ftp/python/2.7.18/Python-2.7.18.tgz

tar xf Python-2.7.18.tgz

cd Python-2.7.18

sudo apt-get install -y libsqlite3-dev

./configure --enable-optimizations

sudo make install -j8

sudo apt-get install -y libgmp-dev libmpc-dev

sudo ln -s /usr/bin/python2.7 /usr/bin/python
```

4.3 SDK 源码获取

CokePi-SDK 的代码被划分为了若干 git 仓库分别进行版本管理，我们可以使用 repo 工具对这些 git 仓库进行统一的下载，提交，切换分支等操作。

4.3.1 下载地址

访问百度网盘资源获取 SDK 源码压缩包:
点击进入链接: https://pan.baidu.com/s/1ZCW9zMjR5cWNObJCrRaNlg?pwd=1234
提取码: 1234
Linux 源码路径：**可乐派网盘->CPG-3568->Linux 源码->rk356x_linux-5.10-gen-rkr9**

提示

由于源码压缩包体积很大，仅在有大量修改的稳定版本时更新，其发布日期可能和镜像发布日期无法对应。当我们在本地解压压缩包以后，只需要借助 Github 做少量更新，即可同步到最新版本。目前采用的是提供离线 patch 的方式更新 SDK

4.3.2 校验源码完整性

```
# 下载源码压缩包后，使用md5sum校验源码压缩包的完整性
md5sum rk356x_linux-5.10-cokepi-rkr9.tar.bz2.* > md5sums.txt

# 对比生成的md5sums.txt文件和网盘rk356x_linux-5.10-cokepi-rkr9.txt文件，MD5值一致即可
diff md5sums.txt rk356x_linux-5.10-cokepi-rkr9.txt
```

4.3.3 解压源码

以下过程以 CokePi_RK3568_Linux_SDK 进行演示，实际文件名称以自己下载的 SDK 为准

```
# 在用户家目录创建 CokePi_SDK 目录
mkdir ~/CokePi_SDK

# 在CokePi_SDK目录创建rk356x_linux-5.10-cokepi-rkr9目录
mkdir ~/CokePi_SDK/rk356x_linux-5.10-cokepi-rkr9

# 将下载的SDK源码移动到CokePi_SDK 目录下
mv rk356x_linux-5.10-cokepi-rkr9.tar.bz2.* ~/CokePi_SDK

# 进入 CokePi_SDK 目录
cd ~/CokePi_SDK

# 解压SDK源码
cat rk356x_linux-5.10-cokepi-rkr9.tar.bz2.* | tar xj -C \
~/CokePi_SDK/rk356x_linux-5.10-cokepi-rkr9

# 解压后得到完整的SDK包，其中包含u-boot、kernel、工具链、buildroot等所有源代码和文档。

# 进入解压后的SDK目录
cd ~/CokePi_SDK/rk356x_linux-5.10-cokepi-rkr9

# 检出.repo 目录下的 git 仓库
.repo/repo/repo sync -l
```

4.4 SDK 更新

提示

后续 SDK 更新、各种不同的输出和接口的功能以文档和补丁形式发布到可乐派网盘

补丁文件如下图所示：

解压后参考补丁说明文档进行打入补丁

rk3568_linux-5.10-cokepi-rkr9_patch-251224

文件名

- build-cokepi-rk3568-linux.md
- changelog
- patch
- rk3568_linux-5.10-cokepi-rkr9_patch-0001.md

4.5 Linux 编译指南

4.5.1 SDK 板级配置准备->RK3568

```
$./build.sh device/rockchip/rk3566_rk3568/rockchip_rk3568_cokepi_lp4_defconfig
#只需执行一次，注意检查output/.config


$./build.sh lunch # 或者直接选择相应的defconfig，注意检查output/.config

选4. rockchip_rk3568_cokepi_lp4_defconfig 即RK3568 cokepi
```

注意

刷机包的 recovery.img 必须使能调试串口 fiq-debugger 编译 kernel+recovery，否则刷机后无法完成 Recovery 初始化。

CFG 和 DTS 切换需修改 device/rockchip/rk3566_rk3568/rockchip_rk3568_cokepi_lp4_defconfig

```
RK_KERNEL_CFG="cokepi_main_defconfig"
RK_KERNEL_DTS_NAME="rk3568-cpg-mipi0"
```

4.5.2 大版本编译步骤

```
$ ./build.sh uboot
$ ./build.sh kernel
$ ./build.sh recovery
$ ./build.sh misc
$ ./build.sh rootfs
$ ./build.sh firmware
```

4.5.3 CPG-3568 系统编译指南

```
完整大版本编译需修改如下内容
$ vi device/rockchip/rk3566_rk3568/rockchip_rk3568_cokepi_lp4_defconfig

RK_KERNEL_CFG="cokepi_main_defconfig"
RK_KERNEL_DTS_NAME="rk3568-cpg-mipi0"
```

1. 内核选项配置修改命令

```
$ cd kernel

$ . mkenv.sh # 设置CROSS_COMPILE环境变量

$ mkcc ARCH=arm64 cokepi_main_defconfig # 根据需要的defconfig指定缺省配置文件名
    字

$ mkcc ARCH=arm64 menuconfig
```

2. CPG-3568 MIPI 输出版本内核编译

```
$ vi device/rockchip/rk3566_rk3568/rockchip_rk3568_cokepi_lp4_defconfig

RK_KERNEL_CFG="cokepi_main_defconfig"
RK_KERNEL_DTS_NAME="rk3568-cpg-mipi0"
```

```
$ ./build.sh kernel
$ ./build.sh recovery # 需要生成recovery.img才运行此命令
```

MIPI1 输出则将对应按 rk3568-cpg-mipi1.dts 配置编译即可

3. CPG-3568 HDMI0 输出版本内核编译

```
HDMI0直接使用MIPI输出版本内核即可支持
```

4. CPG-3568 HDMI1 输出版本内核编译

```
$ vi device/rockchip/rk3566_rk3568/rockchip_rk3568_cokepi_lp4_defconfig

RK_KERNEL_CFG="cokepi_main_defconfig"
RK_KERNEL_DTS_NAME="rk3568-cpg-edphd"
```

4.6 内核更新

提示

当我们完成内核编译后，需要把 `boot.img uboot.img` 等推送到主板以完成内核更新，常见的方式有如下两种。

4.6.1 ADB 推送

使用 adb 命令编写的 bat 脚本将 SDK 编译成功的内核推送到主板

bat 脚本路径: **可乐派网盘->CPG-3568-> 工具软件->adb-linux-bat**

```
@echo off
chcp 65001


REM 更新镜像
adb push ./boot.img /mnt
adb push ./recovery.img /mnt
adb push ./uboot.img /mnt

adb shell "dd if=/mnt/boot.img of=/dev/mmcblk0p3"
adb shell "dd if=/mnt/recovery.img of=/dev/mmcblk0p4"
adb shell "dd if=/mnt/uboot.img of=/dev/mmcblk0p1"

REM 更新wifi.ko
REM SDK中aic8800_bsp.ko所在目录如下
REM kernel/drivers/net/wireless/aic8800/aic8800_bsp
REM SDK中aic8800_fdrv.ko所在目录如下
REM kernel/drivers/net/wireless/aic8800/aic8800_fdrv

adb push ./aic8800_bsp.ko /system/lib/modules
adb push ./aic8800_fdrv.ko /system/lib/modules

REM 重启
adb shell "sleep 5 && echo +150000 > /sys/class/rtc/rtc0/wakealarm"
adb shell "poweroff -f"

echo "升级完成，请重启"
pause
```

常见的 ADB 命令说明

命令	说明
adb reboot loader	切换到 Loader 烧录模式
adb devices	列出所有连接的设备或模拟器
adb push <local> <remote>	将本地文件复制到设备上的指定位置
adb pull <remote> <local>	将设备上的文件复制到本地系统
adb install <apk>	安装 APK 文件到设备（适用于 android 系统）
adb uninstall <package>	卸载指定的应用包（适用于 android 系统）
adb logcat	查看设备的日志输出（调试日志，适用于 android 系统）
adb shell	进入设备的 shell 环境，允许执行命令

Continued on next page

Continued from previous page

命令	说明
adb reboot	重启设备

4.6.2 烧录工具烧写内核

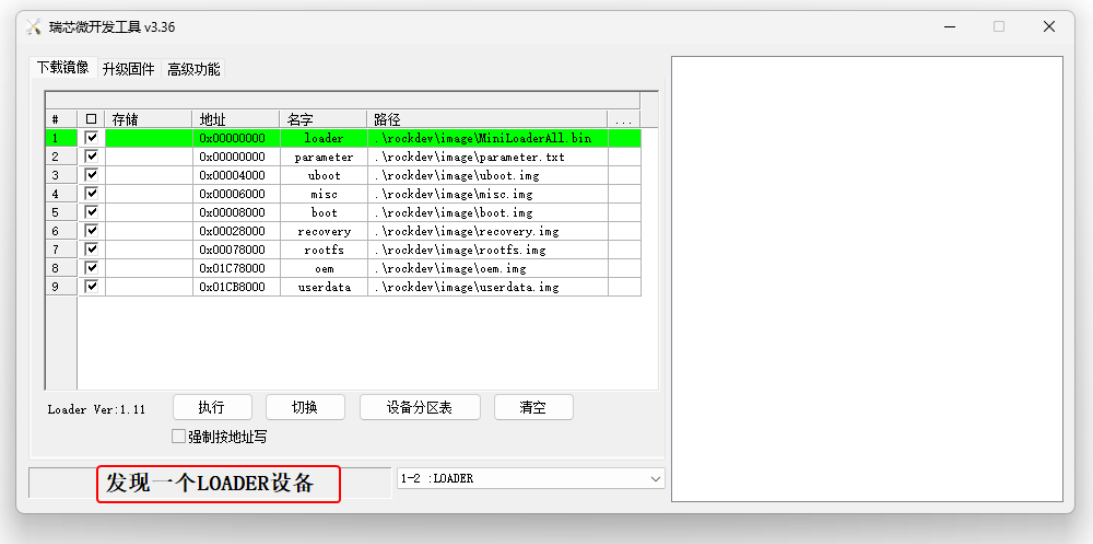
当调试过程中出现内核无法启动时，可以使用烧录工具来完成内核烧写。

- 打开系统镜像包 (固件烧录包)

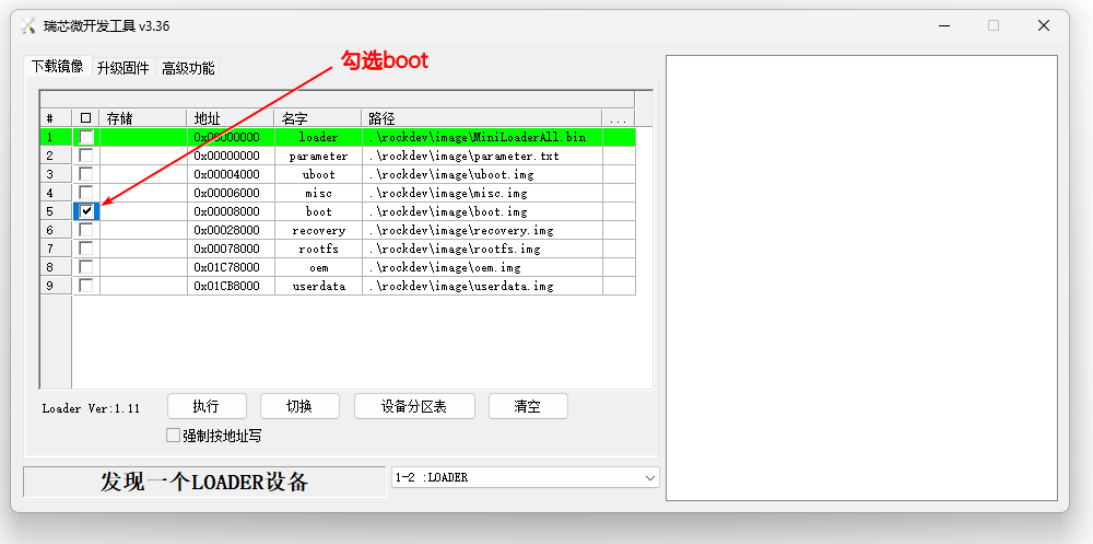
注意

务必使用与主板运行系统相同的系统镜像来进行操作，例如：给运行 Ubuntu 系统的主板进行烧录需要使用 Ubuntu 镜像包

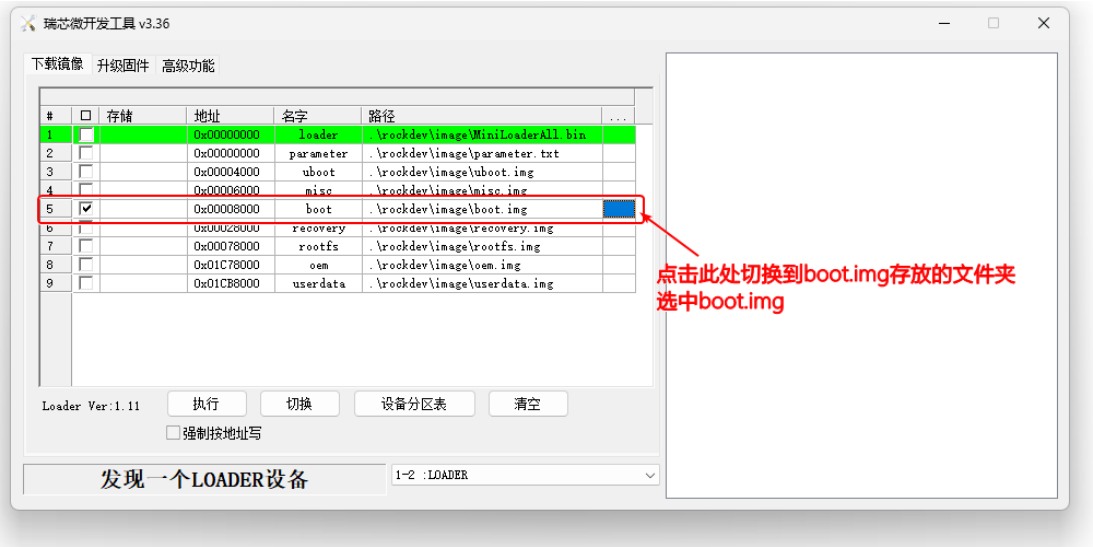
- 进入 loader 烧录模式, 参考2.3.3章节

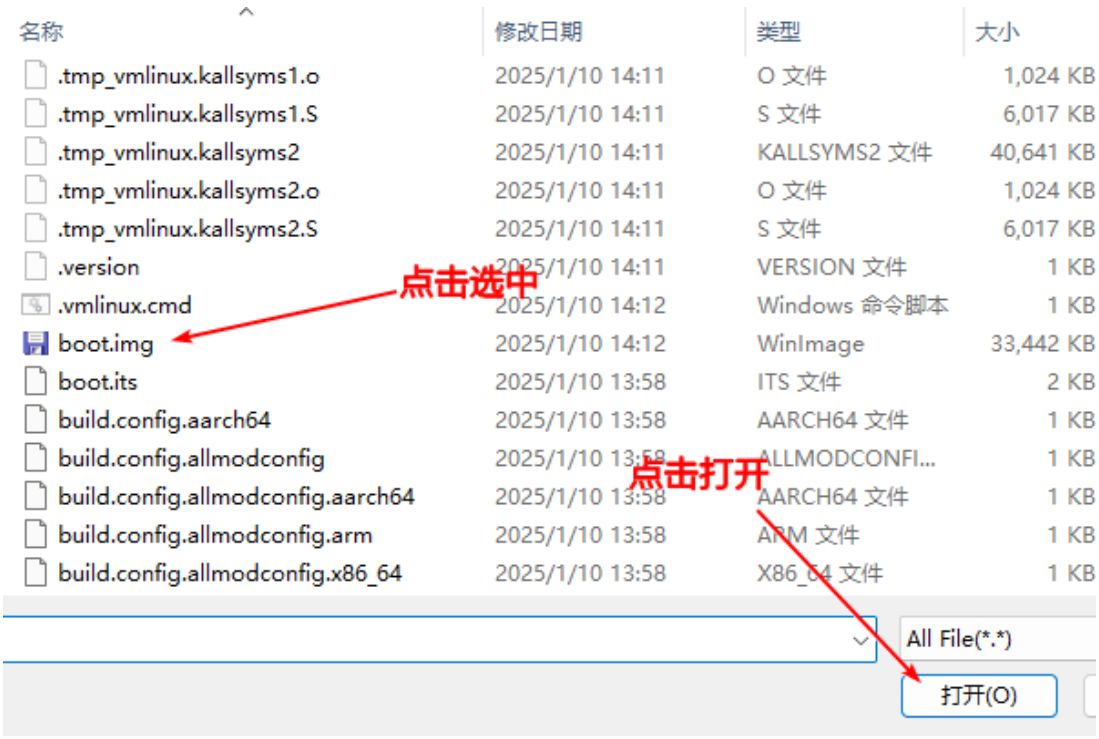


- 先取消所有勾选，再勾选需要烧录的镜像，比如只烧 boot.img，则只需勾选 boot.img

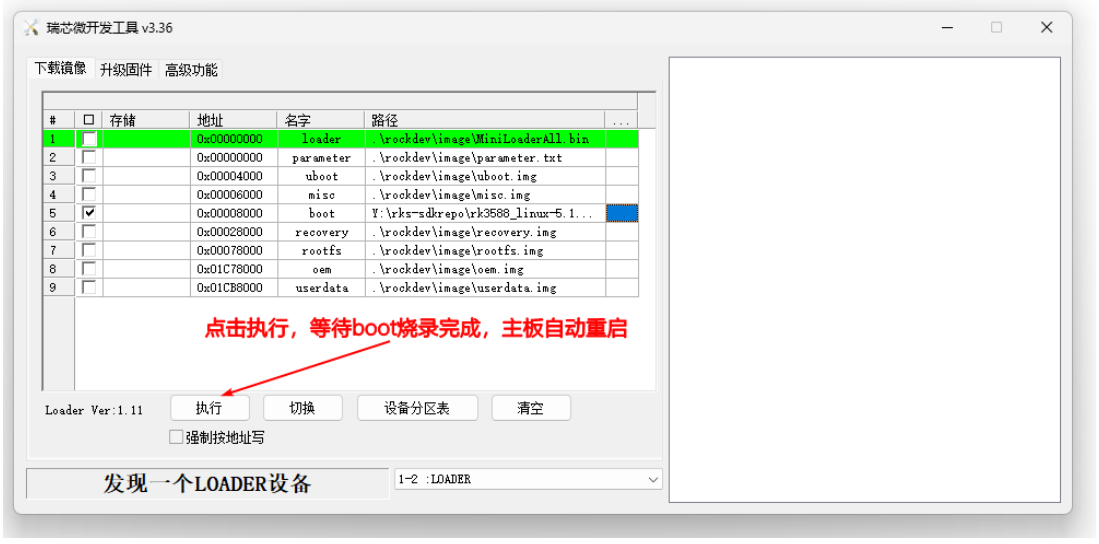


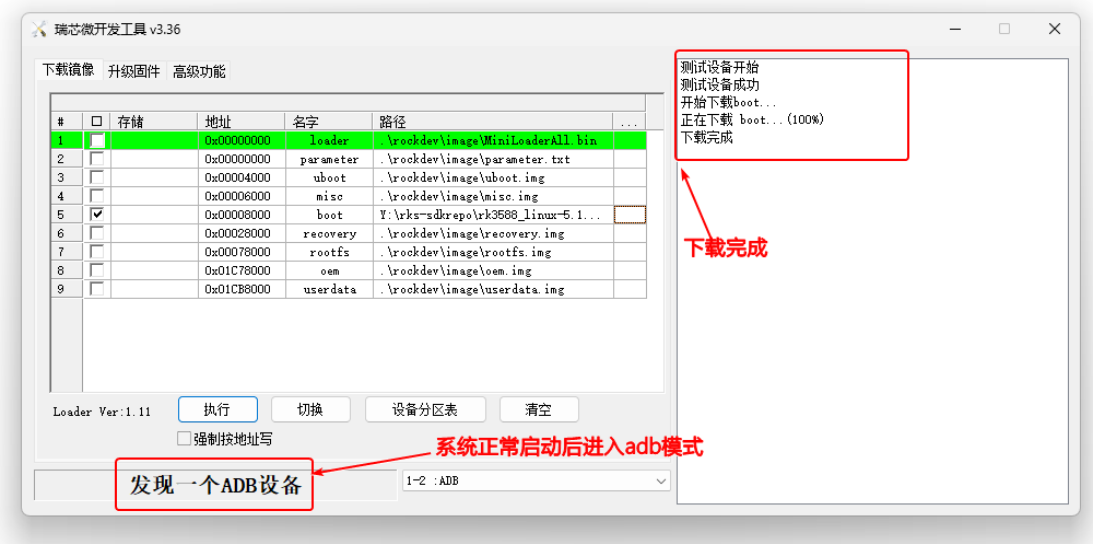
- 切换到 boot.img 存放的目录, 并选中





- 点击执行，烧写完成，自动重启





注意

因为系统中 wifi.ko 是以驱动文件的形式加载的，请留意 wifi.ko 是否发生了改变，如果发生改变需要使用 bat 脚本推送到主板完成更新，否则可能出现异常现象。lsmod 命令可查看驱动是否正常加载，调试阶段也可使用 rm 命令删除/system/lib/modules 目录下的 ko 文件，专注调试

5 Android 11 系统的使用说明

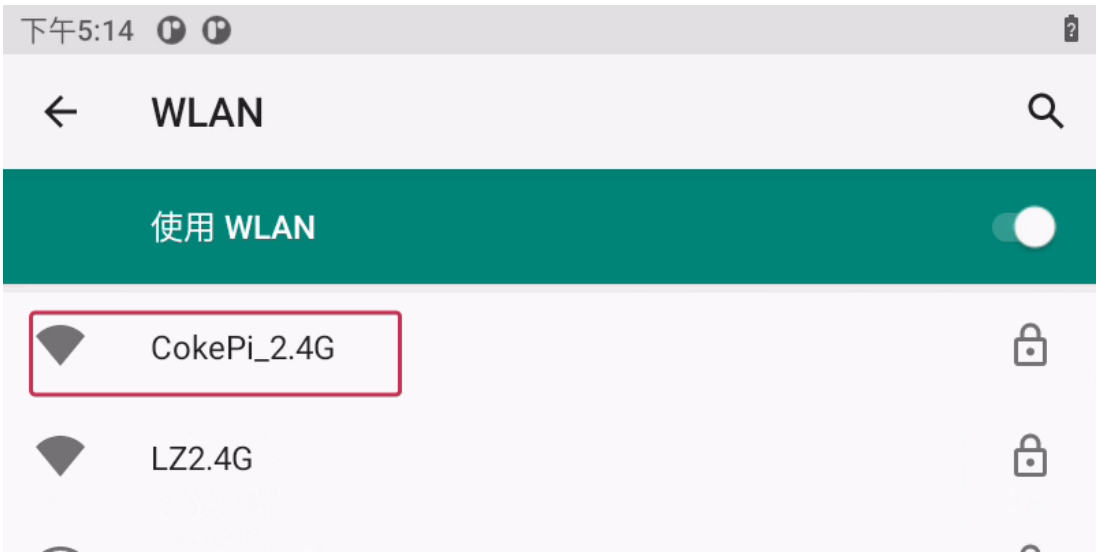
5.1 WiFi 的连接测试方法

- 首先点击进入**设置**

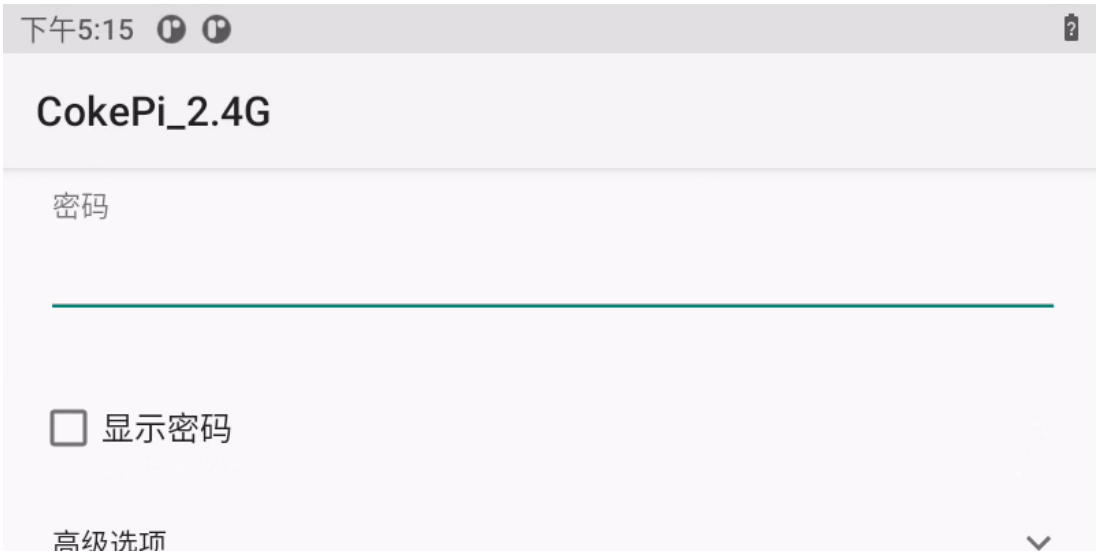
- 然后选择**网络和互联网**

- 然后打开 Wi-Fi 开关，并点击 Wi-Fi 选项

- 打开 Wi-Fi 后就可以扫描到附近的 WiFi 热点了

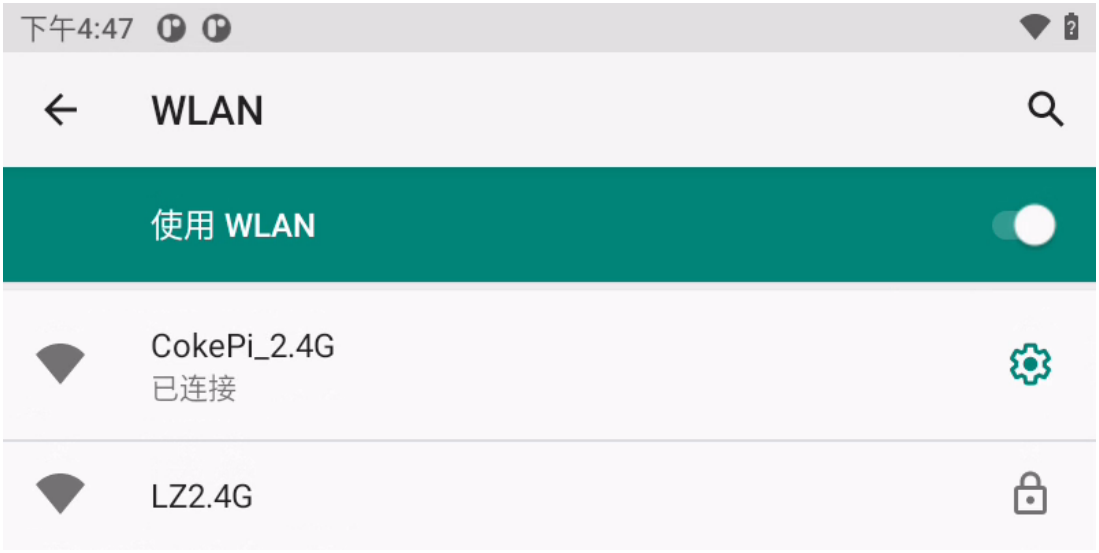


- 然后选择想连接的 Wi-Fi 后会弹出下图所示的密码输入界面



- 然后使用键盘输入 Wi-Fi 对应的密码，再使用鼠标点击虚拟键盘中的回车按钮就会开始连接 Wi-Fi 了

- Wi-Fi 连接成功后的显示如下图所示：



5.2 蓝牙的测试方法

- 首先点击进入**设置**

- 然后选择 **已连接的设备**选项

- 然后点击**与新设备配对**打开蓝牙并开始扫描周围的蓝牙设备

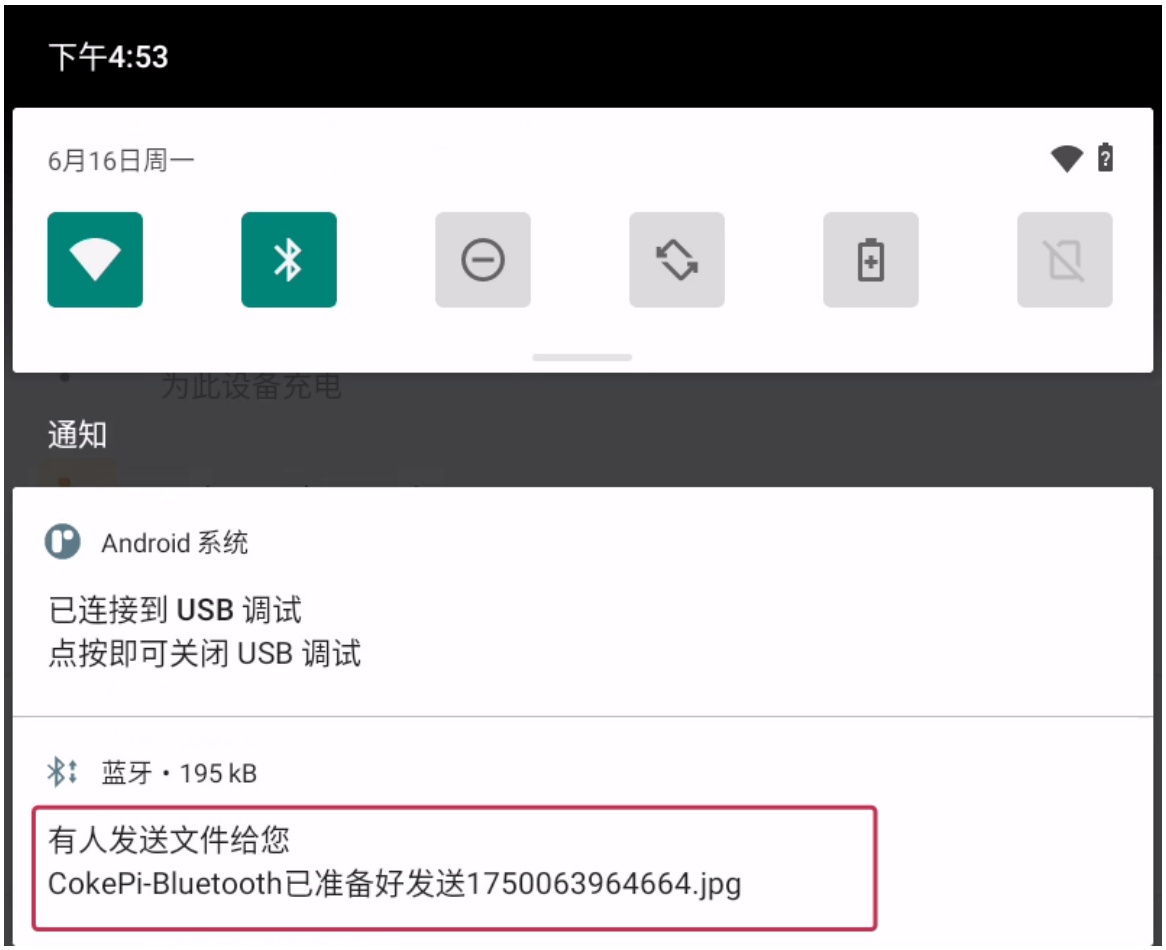
- 搜索到的蓝牙设备会在**可用的设备**下面显示出来

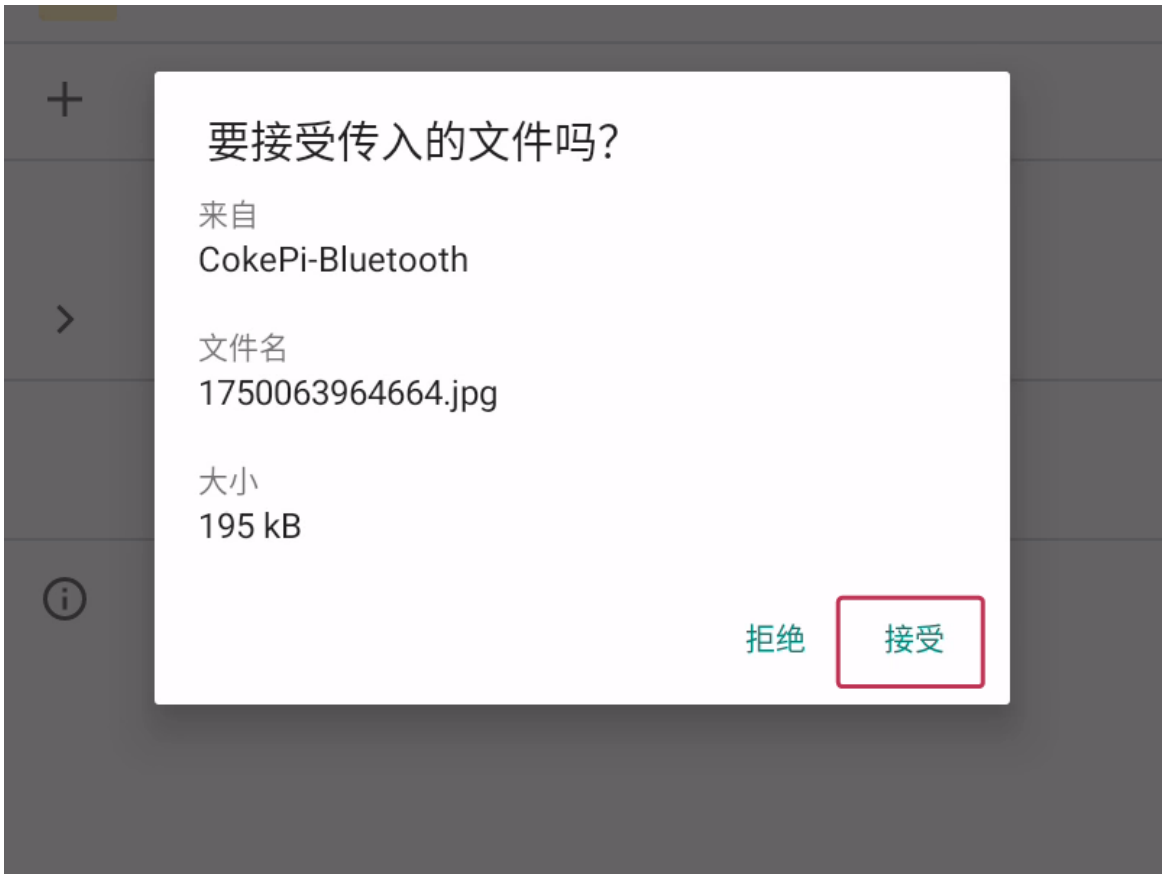
- 然后点击想要连接的蓝牙设备就可以开始配对了，当弹出下面的界面时，请使用鼠标选择**配对**选项

- 这里测试的是主板和安卓手机蓝牙的配置过程，此时在手机上会弹出下面的确认界面，在手机上也点击配对按钮后就会开始配对过程

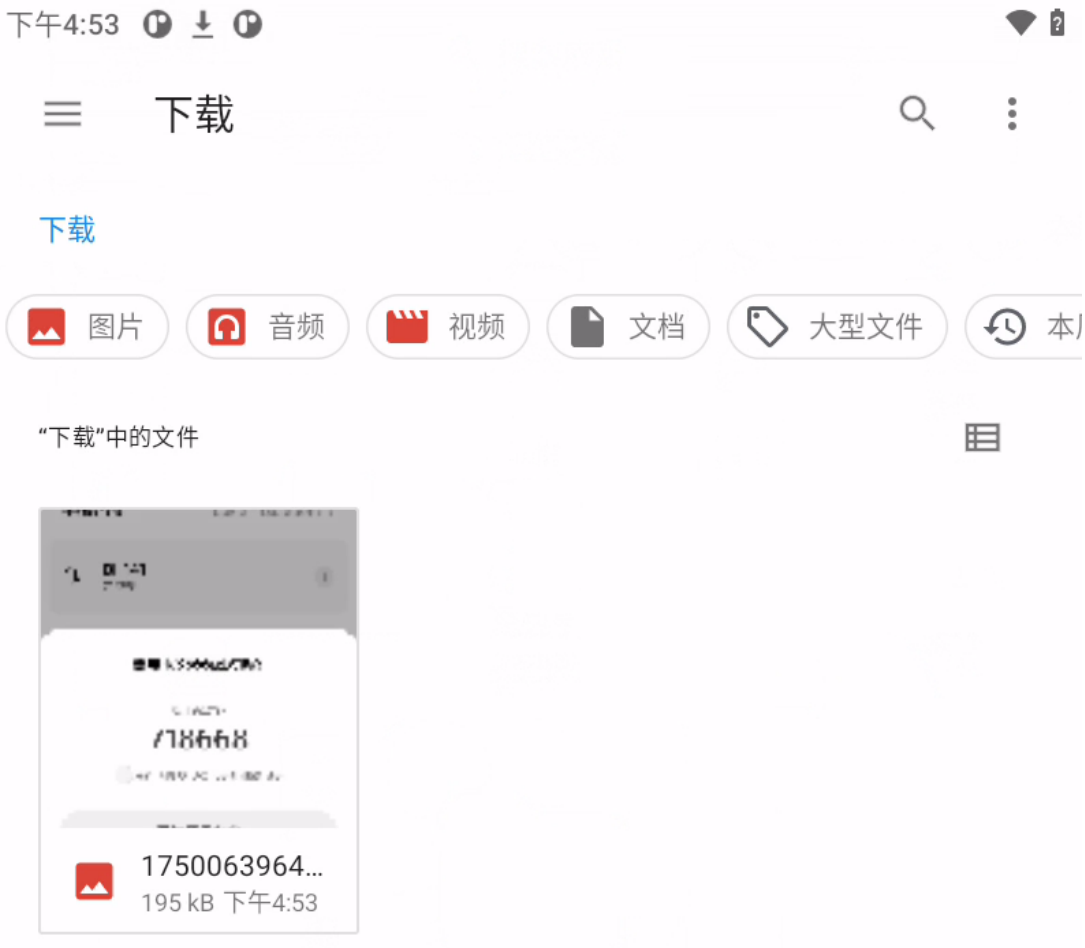
- 配对完成后，可以看到如下图所示的已配对的蓝牙设备

- 此时可以使用手机蓝牙给主板发送一张图片，发送后，在主板的安卓系统中可以看到下面的确认界面，然后点击**接受**就可以开始接收手机发过来的图片了





- 主板 Android 系统蓝牙接收到的图片可以在**文件管理器**中打开 Download 目录查看



5.3 8 寸 MIPI 屏幕的使用方法

- 准备需要的配件

- 使用从我司采购的 MIPI 屏幕 (型号:xmz080cs6)，只要线序连接正确，连线牢固，刷标准系统默认即可点屏
- 使用其他型号屏幕，需保证线序正确、屏幕参数正确、屏幕供电电压一致等条件均满足，才可正常点屏

注意

连接好屏线后，请以板子丝印为准仔细检查线序再通电，MIPI 屏比较脆弱，接反会烧屏。以防万一，请勿参考屏线上面的丝印，要以板子丝印和屏幕丝印为准

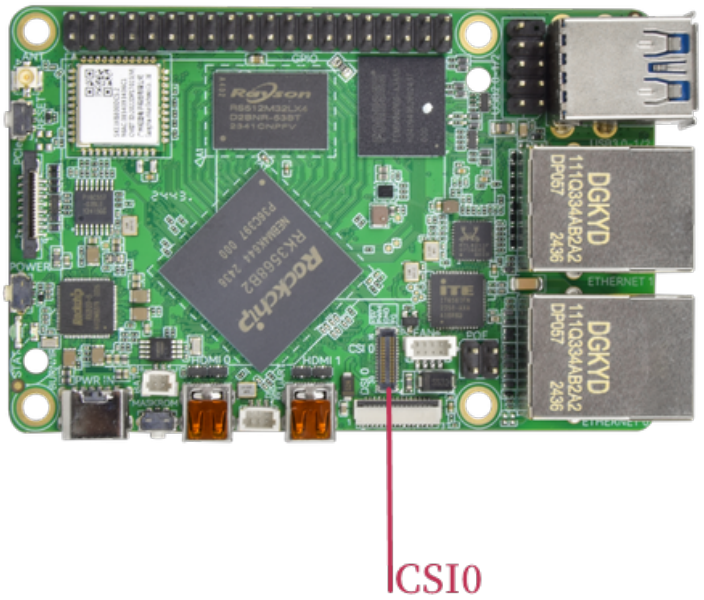
5.4 OV13850 和 OV13855 MIPI 摄像头的测试方法

目前可乐派支持两款 MIPI 摄像头，OV13850 和 OV13855，具体的图片如下所示：

1300 万 MIPI 接口的 OV13850 摄像头

1300 万 MIPI 接口的 OV13855 摄像头

CPG-3568 主板上有 1 个摄像头接口，我们定义 CSI0 的位置如下图所示：



摄像头插在主板的 CSI0 接口的方法如下所示：

连接好摄像头到开发板上后，我们可以使用下面的方法来测试下摄像头：在桌面中打开相机 APP

然后就能看到摄像头的预览画面了

5.5 40-Pin 接口测试

40-Pin 接口引脚图 (更详细的请参考硬件原理图)

引脚位置	定义	引脚名称	功能
1	3V3		输出 3.3V
2	5V		输出 5V
3	I2C2_SDA	GPIO4_B4_d=#140	默认 GPIO，可配置为 I2C2 或 CAN2
4	5V		输出 5V
5	I2C2_SCL	GPIO4_B5_d=#141	默认 GPIO，可配置为 I2C2 或 CAN2
6	GND		
7	GPIO3	GPIO3_B0_d=#104	默认 GPIO
8	UART0_TX	GPIO0_C1_d=#17	默认 UART0，可配置为 GPIO 或 PWM
9	GND		
10	UART0_RX	GPIO0_C0_d=#16	默认 UART0，可配置为 GPIO 或 PWM
11	UART3_RX	GPIO3_C0_d=#112	默认 UART3，可配置为 GPIO 或 PWM
12	I2S3_SCLK	GPIO4_C3_d =#147	默认 GPIO，可配置为 I2S3 或 CAN1
13	UART3_TX	GPIO3_B7_d=#111	默认 UART3，可配置为 GPIO 或 PWM
14	GND		
15	UART5_TX	GPIO3_C2_d=#114	默认 UART5，可配置为 GPIO
16	UART4_TX	GPIO3_B2_d=#106	默认 UART4，可配置为 GPIO

Continued on next page

Continued from previous page

引脚位置	定义	引脚名称	功能
17	3V3		3.3V
18	UART4_RX	GPIO3_B1_d=#105	默认 UART，可配置为 GPIO
19	SPI0_MOSI	GPIO2_D1_d=#89	默认 GPIO，可配置为 SPI0
20	GND		
21	SPI0_MISO	GPIO2_D0_d=#88	默认 GPIO，可配置为 SPI0
22	UART5_RX	GPIO3_C3_d=#115	默认 UART，可配置为 GPIO
23	SPI0_CLK	GPIO2_D3_d=#91	默认 GPIO，可配置为 SPI0
24	SPI0_CS0	GPIO2_D2_d=#90	默认 GPIO，可配置为 SPI0
25	GND		
26	GPIO3_A7	GPIO3_A7=#103	默认 GPIO
27	I2C5_SDA	GPIO3_B4_d=#108	默认 GPIO，可配置为 I2C5
28	I2C5_SCL	GPIO3_B3_d=#107	默认 GPIO，可配置为 I2C5
29	GPIO3_D2	GPIO3_D2_d=#122	默认 GPIO
30	GND		
31	I2S3_MCLK	GPIO4_C2_d=#146	默认 GPIO，可配置为 I2S3 或 CAN1
32	PWM14	GPIO3_C4_d=#116	默认 PWM14, 可配置为 GPIO
33	PWM15	GPIO3_C5_d=#117	默认 GPIO，可配置为 PWM15
34	GND		
35	I2S3_LRCK	GPIO4_C4_d=#148	默认 GPIO，可配置为 I2S3
36	UART8_TX	GPIO2_D7_d=#95	默认 UART8，可配置为 GPIO
37	UART8_RX	GPIO3_A0_d=#96	默认 UART，可配置为 GPIO
38	I2S3_SDIO	GPIO4_C6_d=#150	默认 GPIO，可配置为 I2S3
39	GND		
40	I2S3_SDO	GPIO4_C5_d=#149	默认 GPIO，可配置为 I2S3

5.5.1 GPIO 测试

最常见的读写 GPIO 方式就是用 GPIO sysfs interface，通过操作/sys/class/gpio 目录下的 export 、unexport 、gpio{N}/direction, gpio{N} /value （用实际引脚号替代 {N}）等文件实现的。

GPIO 举例计算：

引脚位置	引脚名称	控制器	端口号	索引号	计算结果
7	GPIO3_B0=#104	3	B	0	104(32x3+8x1+0)

```
bank = 3;
// GPIO3_B0 => 3, bank ? [0,4]
group = 1;
// GPIO3_B0 => 1, group ? {(A=0), (B=1), (C=2), (D=3)}
X = 1;
```

```
// GPIO3_B0 => 0, X ? [0,7]
number = group * 8 + X = 1 * 8 + 0 = 8;
pin = bank * 32 + number = 3 * 32 + 8 = 104;
```

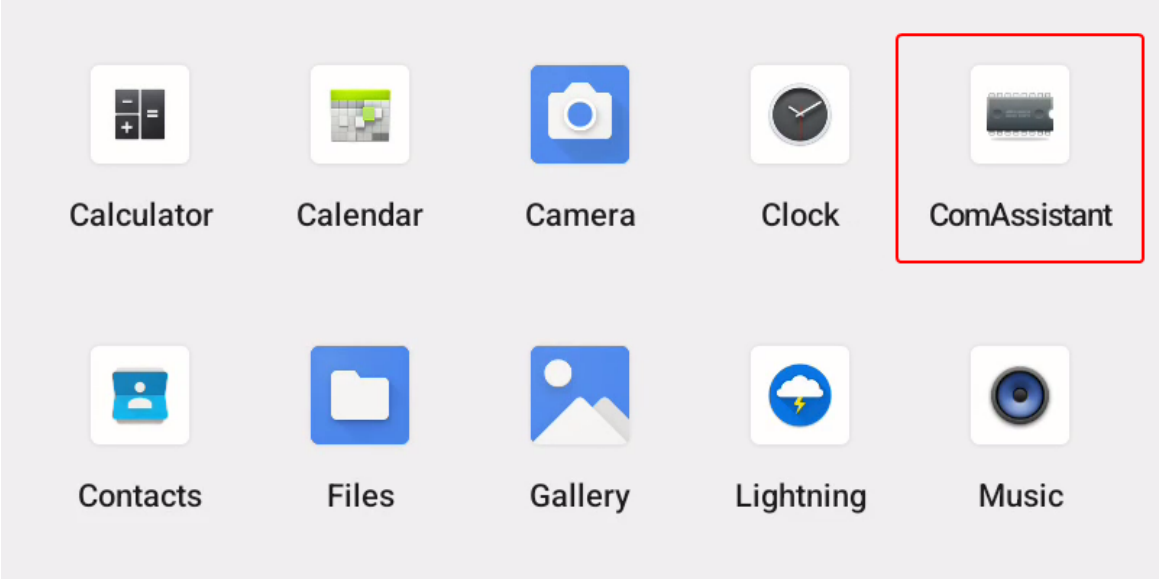
```
# 以下所有操作均需要打开管理者权限使用
su
# 使能引脚 GPIO3_B0
echo 104 > /sys/class/gpio/export

# 设置引脚为输入模式
echo in > /sys/class/gpio/gpio104/direction

# 读取引脚的值
cat /sys/class/gpio/gpio104/value

# 设置引脚为输出模式
echo out > /sys/class/gpio/gpio104/direction

# 设置引脚为低电平
echo 0 > /sys/class/gpio/gpio104/value

# 设置引脚为高电平
echo 1 > /sys/class/gpio/gpio104/value

# 复位引脚
echo 104 > /sys/class/gpio/unexport
```

5.5.2 UART 测试

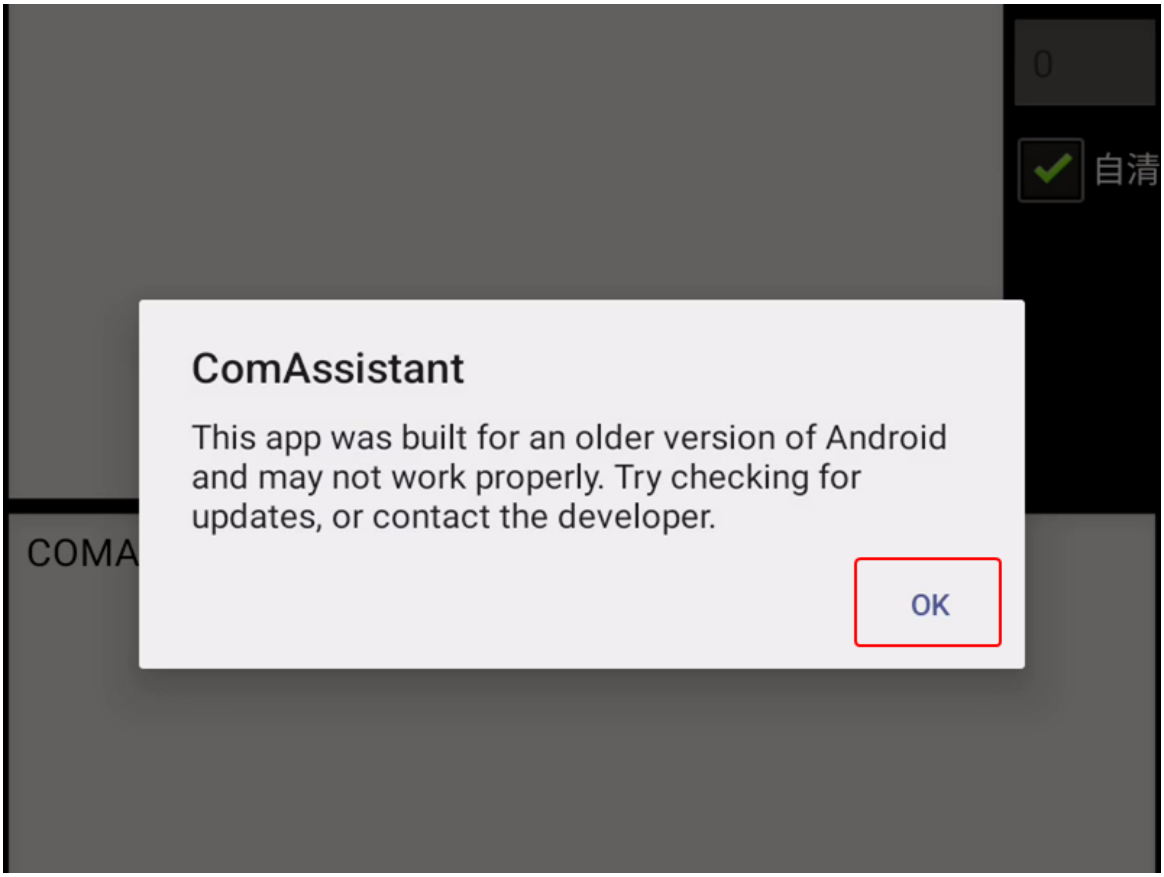
下载安装串口助手.apk，电脑 CMD 命令行终端连接 adb 执行如下命令安装：
网盘路径: **可乐派网盘->CPG-3568-> 工具软件-> 串口助手.apk**

```
PS C:\Users\Administrator> adb install .\Desktop\串口助手.apk
Success
```

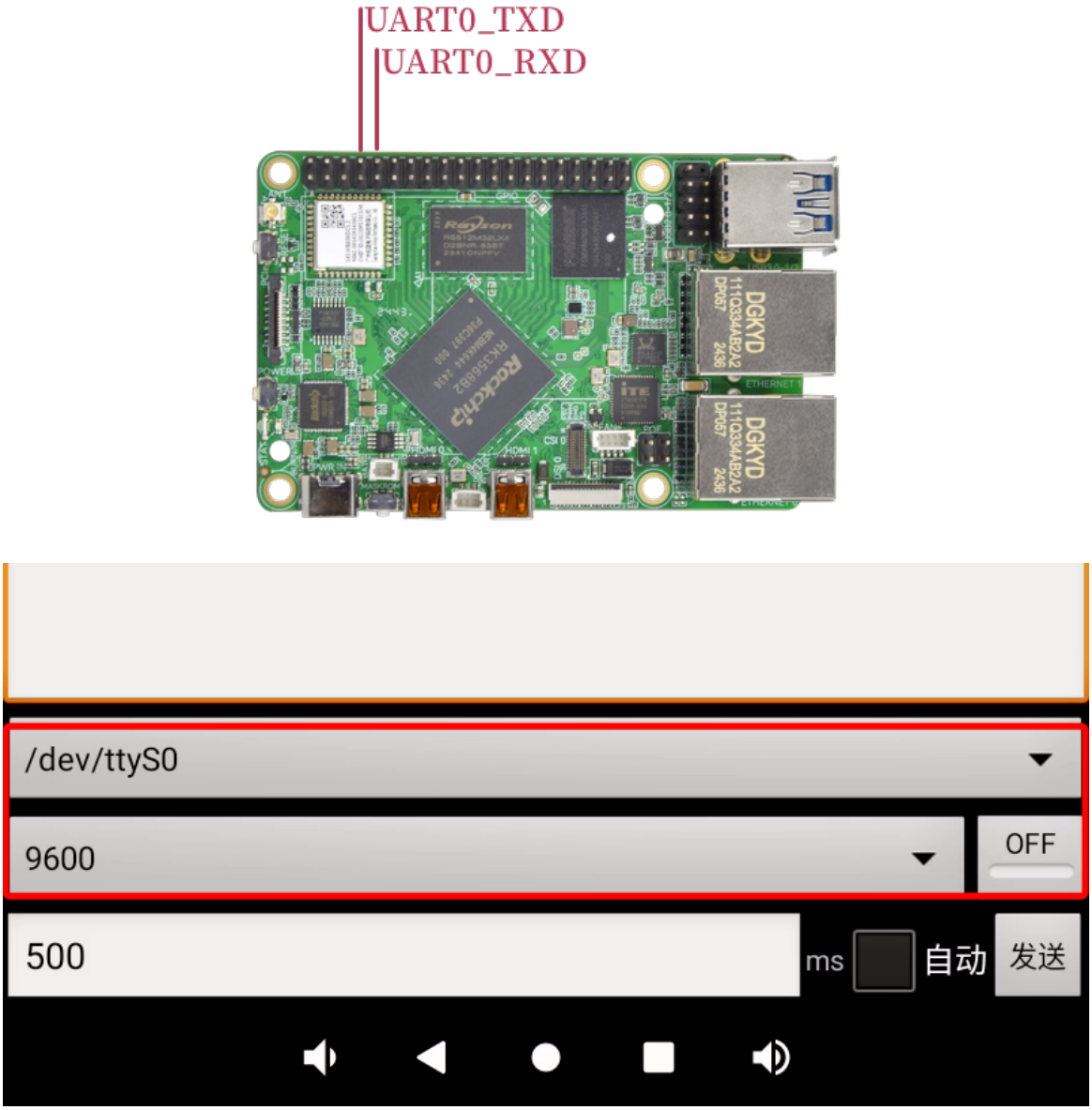
安卓界面点击图标打开



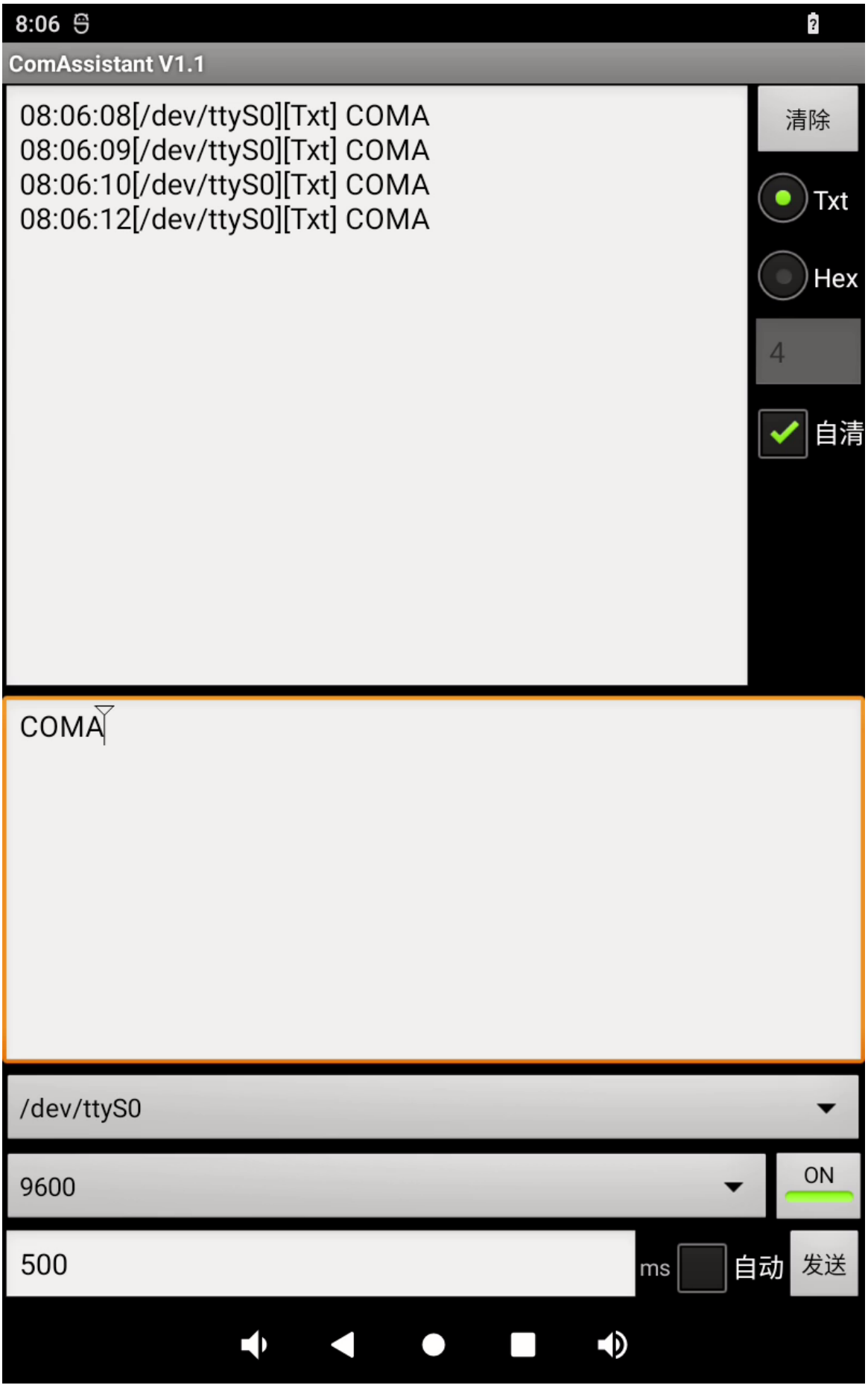
点击 OK



接着在下拉框中选择想要设置的波特率和设备节点，然后点击右侧 OFF 按钮，然后使用杜邦线短接 uart0 的 RXD 和 TXD 引脚



然后可以在下面的发送编辑框中输入一段字符，点击发送按钮开始发送，如果一切正常，接收框内会显示已接收到的字符串


8:06
ComAssistant V1.1
08:06:08[/dev/ttyS0][Txt] COMA
08:06:09[/dev/ttyS0][Txt] COMA
08:06:10[/dev/ttyS0][Txt] COMA
08:06:12[/dev/ttyS0][Txt] COMA
清除
Txt
Hex
4
自清
COMA
/dev/ttyS0
9600
ON
500
ms
自动
发送

6 Android SDK 使用说明

6.1 SDK 开发环境搭建

CokePi-SDK 是基于 Ubuntu 22.04 系统开发测试的，为了不必要的麻烦，我们推荐用户使用 Ubuntu 22.04 系统。磁盘空间建议大于 300GB，内存建议大于 4GB。

6.1.1 安装 SDK 依赖的软件包

```
# 安装 SDK 构建所需要的软件包
# 整体复制下面内容到终端中安装

sudo apt install git ssh make gcc libssl-dev liblz4-tool u-boot-tools curl expect \
g++ patchelf chrpath gawk texinfo chrpath diffstat binfmt-support \
qemu-user-static live-build bison flex fakeroot cmake gcc-multilib \
g++-multilib unzip device-tree-compiler python3-pip libncurses5-dev \
python3-pyelftools dpkg-dev
```

6.1.2 安装 repo

```
mkdir ~/bin

curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo

chmod a+x ~/bin/repo

echo PATH=~/bin:$PATH >> ~/.bashrc

source ~/.bashrc
```

- 执行完上面的命令后我们来验证 repo 是否安装成功能正常运行。

```
repo --version

# 返回以下信息
# 返回的信息根据Ubuntu版本的不同略有差异
<repo not installed>
repo launcher version 2.50
      (from /home/teamhd/bin/repo)
git 2.34.1
Python 3.10.12 (main, Feb 4 2025, 14:57:36) [GCC 11.4.0]
OS Linux 6.8.0-52-generic (#53~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Wed Jan 15
    19:18:46 UTC 2)
CPU x86_64 (x86_64)
Bug reports: https://issues.gerritcodereview.com/issues/new?component=1370071
```

6.1.3 安装 python2.7

```
cd ~

wget https://www.python.org/ftp/python/2.7.18/Python-2.7.18.tgz

tar xf Python-2.7.18.tgz

cd Python-2.7.18

sudo apt-get install -y libsqlite3-dev

./configure --enable-optimizations

sudo make install -j8

sudo apt-get install -y libgmp-dev libmpc-dev

sudo ln -s /usr/bin/python2.7 /usr/bin/python
```

6.2 SDK 源码获取

CokePi-SDK 的代码被划分为了若干 git 仓库分别进行版本管理，我们可以使用 repo 工具对这些 git 仓库进行统一的下载，提交，切换分支等操作。

6.2.1 下载地址

访问百度网盘资源获取 SDK 源码压缩包:
点击进入链接: https://pan.baidu.com/s/1ZCW9zMjR5cWNObJCrRaNlg?pwd=1234
提取码: 1234
安卓源码路径：**可乐派网盘->CPG-3568-> 安卓源码->rk356x_android-11.0-mid-rkr12**

提示

由于源码压缩包体积很大，仅在有大量修改的稳定版本时更新，其发布日期可能和镜像发布日期无法对应。当我们在本地解压压缩包以后，只需要借助网盘最新补丁做少量更新，即可同步到最新版本。目前采用的是提供离线 patch 的方式更新 SDK。

6.2.2 校验源码完整性

```
# 下载源码压缩包后，使用md5sum校验源码压缩包的完整性
md5sum rk356x_android-11-cokepi-rkr12.tar.bz2.* > md5sums.txt

# 对比生成的md5sums.txt文件和网盘rk356x_android-11-cokepi-rkr12.txt文件，MD5值一致即可
diff md5sums.txt rk356x_android-11-cokepi-rkr12.txt
```

6.2.3 解压源码

以下过程以 CokePi_RK3588_Android_SDK 进行演示，实际文件名称以自己下载的 SDK 为准

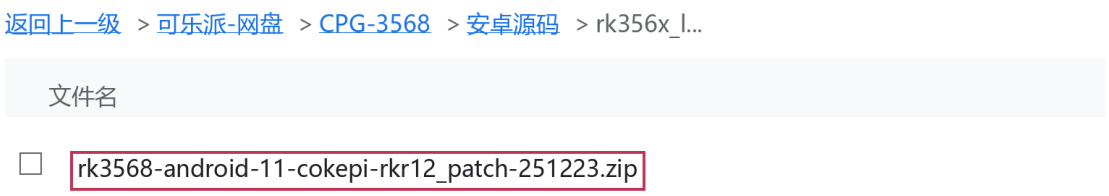
```
# 在用户家目录创建CokePi_SDK目录
mkdir ~/CokePi_SDK

# 在CokePi_SDK目录创建rk356x_android-11-cokepi-rkr12目录
mkdir ~/CokePi_SDK/rk356x_android-11-cokepi-rkr12

# 将下载的SDK源码移动到CokePi_SDK 目录下
mv rk356x_android-11-cokepi-rkr12.tar.bz2.* ~/CokePi_SDK

# 进入 CokePi_SDK 目录
cd ~/CokePi_SDK

# 解压SDK源码
cat rk356x_android-11-cokepi-rkr12.tar.bz2.* | tar xj -C \
~/CokePi_SDK/rk356x_android-11-cokepi-rkr12

# 解压后得到完整的SDK包rk356x_android-11-cokepi-rkr12，其中包含u-boot、kernel、工具链、buildroot等所有源代码和文档。

# 进入解压后的SDK目录
cd ~/CokePi_SDK/rk356x_android-11-cokepi-rkr12

# 检出.repo 目录下的 git 仓库
.repo/repo/repo sync -l
```

6.3 SDK 更新补丁

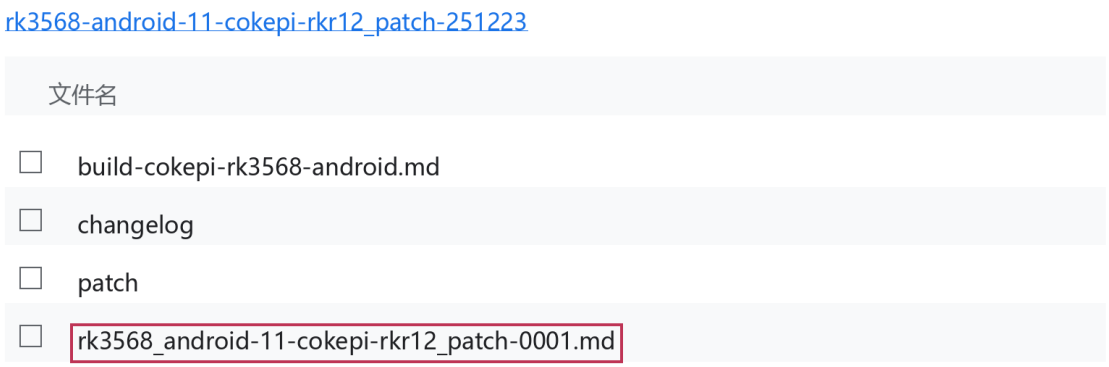
提示

SDK 更新、各种不同的输出和接口的功能以文档和补丁形式发布到可乐派网盘

补丁文件如下图所示：



解压后参考补丁说明文档进行打入补丁



6.4 CPG-3568 系统编译指南

6.4.1 RK3568 编译配置

```
$ . build/envsetup.sh

$ lunch rk3568_r-userdebug

$ ./build.sh -AUCK # 如果需要生成OTA包则-AUKo
```

6.4.2 编译报错及解决办法

```
make: ./scripts/mkimg: Permission denied make: *** [arch/arm64/Makefile:203:
    rk3568-cpg-mipi0.img] Error 127 #### failed to build some targets (02:10 (mm
    :ss)) #### Build kernel failed!

在rk356x_android-11-cokepi-rkr12目录下执行如下命令后，重新执行编译命令:
chmod +x ./kernel/scripts/mkimg
```

注：源码补丁已修复此问题，打入补丁后不会再出现此问题。

6.4.3 编译配置参数文件设定

```
U-boot、内核的配置和dts文件通过U-boot、内核的配置和dts文件通过device/rockchip/rk356x/
    rk3568_r/BoardConfig.mk来指定来指定

PRODUCT_UBOOT_CONFIG := rk3568
PRODUCT_KERNEL_DTS := rk3568-cpg-mipi0
PRODUCT_KERNEL_CONFIG := cokepi_main_defconfig
```

6.4.4 U-Boot 编译指南

```
U-boot编译命令RK3568:
$ cd u-boot; make mrproper && ./make.sh rk3568
```

6.4.5 内核编译指南

```
完整安卓大版本编译需修改如下内容:
$ vi device/rockchip/rk356x/rk3568_r/BoardConfig.mk

PRODUCT_KERNEL_DTS := rk3568-cpg-mipi0
PRODUCT_KERNEL_CONFIG := cokepi_main_defconfig
```

6.4.6 内核选项配置修改命令

```
$ cd kernel
$ make ARCH=arm64 cokepi_main_defconfig # 根据需要的defconfig指定缺省配置文件名字

$ mkcl ARCH=arm64 menuconfig
```

6.4.7 MIPI 0 | MIPI1 输出版本内核编译

```
- kernel/arch/arm64/boot/dts/rockchip/rk3568-cpg-mipi0.dts

$ cd kernel # 由PRODUCT_KERNEL_DTS和PRODUCT_KERNEL_CONFIG指定相应的编译配置!
```

```
$ cp arch/arm64/configs/cokepi_main_defconfig .config
$ ./mkboot.sh rk3568-cpg-mipi0.img # 在kernel目录下生成boot.img

$ ./mkrecovery.sh rk3568-cpg-mipi0.img # 需要生成recovery.img才运行此命令
```

MIPI1 输出则将对应按 rk3568-cpg-mipi1.dts 配置编译即可

6.4.8 HDMI0 输出版本内核编译

```
直接使用MIPI输出版本内核即可支持
```

6.4.9 HDMI1 输出版本内核编译

```
- kernel/arch/arm64/boot/dts/rockchip/rk3568-cpg-edphd.dts

$ cd kernel # 由PRODUCT_KERNEL_DTS和PRODUCT_KERNEL_CONFIG指定相应的编译配置!
$ cp arch/arm64/configs/cokepi_main_defconfig .config

$ ./mkboot.sh rk3568-cpg-edphd.img # 在kernel目录下生成boot.img

$ ./mkrecovery.sh rk3568-cpg-edphd.img # 需要生成recovery.img才运行此命令
```

6.5 内核更新

6.5.1 ADB 推送

使用 adb 命令编写的 bat 脚本将 SDK 编译成功的内核推送到主板

bat 脚本路径: **可乐派网盘->CPG-3568-> 工具软件->adb-android-bat**

```
@echo off
chcp 65001

REM 更新镜像
adb root
adb remount
adb push ./boot.img /mnt/boot.img
abd push ./recovery.img /mnt/recovery.img
adb push ./uboot.img /mnt/uboot.img

adb shell "dd if=/mnt/boot.img of=/dev/block/by-name/boot"
adb shell "dd if=/mnt/recovery.img of=/dev/block/by-name/recovery"
adb shell "dd if=/mnt/uboot.img of=/dev/block/by-name/uboot"
adb shell "sleep 5 && echo +150000 > /sys/class/rtc/rtc0/wakealarm"
adb shell "reboot -p"

echo "升级完成，请重启"
pause
```

7 附录

7.1 用户手册更新历史

版本	日期	更新说明
v1.2	2025-12-29	更新网盘路径、增加打补丁说明、默认 HDMI 显示
v1.1	2025-06-19	修改供电电流大小标注
v1.0	2025-06-17	初始版本

7.2 镜像更新历史

日期	更新说明
2025-06-17	初始版本

Published March 2025